

DEVELOPMENT STORY

AI 에이전트 팀으로 차세대 추천 시스템을 만들기까지

데스크톱 GPU 1대, 3인 팀, 그리고 AI 협업의 기록

"AI가 코드를 쓰지만,
설계 판단은 사람이 한다."

인프라 예산 없이, 데스크톱 GPU 1대와 AI 에이전트들의 조합으로
13-task, 7-expert PLE+adaTT 추천 시스템을 구축한 과정을 기록한다.

AI Ops PLE Platform

프로젝트 배경

팀 구성과 제약조건

프로젝트 팀은 3명이었다. 데이터사이언티스트 겸 PM/리드 아키텍트 1명과 엔지니어 2명. 전용 인프라 예산은 없었고, 개발에 사용할 수 있는 GPU는 로컬 PC에 장착된 데스크톱용 RTX 4070 (12GB VRAM) 1대가 전부였다.

제약조건 요약

- **팀 규모:** 3명 (PM/리드 아키텍트/데이터사이언티스트 1 + 엔지니어 2)
- **인프라:** 전용 GPU 서버 없음. RTX 4070 12GB 1대 (로컬)
- **예산:** 인프라 구매 예산 없음. AWS SageMaker spot 인스턴스 활용
- **일정:** 기존 ALS 기반 추천 시스템을 대체할 차세대 시스템 구축

인프라 제약의 현실

조직으로부터 받은 지원은 사실상 전무했다. GPU가 필요하다고 요청해도 “어쩔 수 없다”는 답변뿐이었고, AI 도구 구독료(Claude Code, Gemini, Cursor), AWS 클라우드 비용(SageMaker Spot 인스턴스, S3 스토리지), 부속기기, 식비 등 프로젝트에 소요된 모든 비용은 PM/리드 아키텍트 개인 자금으로 충당했다.

데이터 수집 환경도 열악했다. Spark나 Impala로의 전환 요청이 거부되어 HIVE 기반의 병목 환경에서 작업해야 했고, 이를 극복하기 위해 병렬 쿼리 로직을 직접 설계하여 네트워크 대역폭을 최대한 활용했다.

작업 공간은 열순환이 되지 않는 서버실 옆 공간으로, 적절한 냉방도 제공되지 않았다.

함께 일한 팀원 2명은 정식 계약이 아닌 청년 자문위원 자격의 서포터즈로, 대학 졸업 후 취업을 준비하는 과정에서 프로젝트에 참여했다.

이 모든 제약이 오히려 설계의 개성을 발현시켰다.

생물의 진화에서 선택압(selection pressure)이 종의 특화를 이끌듯, 자원의 제약은 설계에 선택압으로 작용했다.

12GB VRAM이라는 한계는 “파라미터를 늘려서 성능을 올리자”는 안이한 접근을 원천적으로 차단했고, 대신 “구조적 귀납 편향으로 표현력을 확보하자”는 방향으로 진화를 강제했다. 이중 전문가 7개가 각자 다른 수학적 관점을 인코딩하는 설계, 11개 학문 분야에서 구조적 동형사상을 빌려온 피쳐 엔지니어링, FP32에서도 돌아가도록 각 전문가를 경량화한 선택 — 이 모든 것이 제약 속에서 나온 적응의 결과물이다.

대형 GPU 클러스터가 있었다면 이 아키텍처는 탄생하지 않았을 것이다. Transformer 기반 대형 전문가를 7개 쌓고 파라미터로 밀어붙이는 평범한 접근을 택했을 가능성이 높다. 제약이 없었다면 개성도 없었다.

온프레임 시스템 규모

온프레임 시스템은 단순한 프로토타입이 아니라 프로덕션 규모의 시스템이었다. 80개 이상의 Airflow DAG, Champion-Challenger 모델 경쟁, 주간 자동 재학습, 734D 피쳐 텐서, 18개 동시 태스크(AWS 벤치마크 버전에서는 결정론적 리키지/중복 태스크 5개 제거 후 13개), 62개 데이터 테이블 인제스션을 포함한다. 이 규모의 시스템을 3명이 구축한 것 자체가 AI 에이전트 활용의 결과물이다.

프로젝트 목표

기존 금융 상품 추천 시스템은 ALS (Alternating Least Squares) 기반의 협업 필터링이었다. 이를 PLE (Progressive Layered Extraction) + adaTT (Adaptive Task Transfer) 아키텍처 기반의 멀티태스크 딥러닝 추천 시스템으로 교체하는 것이 목표였다. 13개 태스크를 7개 전문가 네트워크가 처리하며, 태스크 간 관계를 명시적으로 모델링하는 구조다.

아키텍처 의사결정 여정

최종 아키텍처에 도달하기까지 여러 후보를 탐색하고 기각하는 과정을 거쳤다.

기존 시스템은 ALS 기반 협업 필터링이었다. 첫 번째 대안으로 **Black-Litterman 모델**을 검토했다. 포트폴리오 이론에서 온 이 모델은 전문가 의견(뷰)과 시장 균형을 베이지안 업데이트로 결합한다. 그러나 사후분포로 혼합된 결과에서 “어떤 모델이 얼마나 기여했는가”를 분리할 수 없었다. 금융 현장에서는 고객, 영업점, 규제기관 모두에게 추천 근거를 설명해야 하는데, 베이지안 업데이트가 개별 모델의 기여를 불투명하게 만들어 비즈니스 설명이 불가능했다.

두 번째 대안은 **모델 앙상블**이었다. N개 모델을 독립적으로 학습시키고 결과를 결합하는 방식이다. 그러나 N개의 관리 포인트, N배의 서버 비용이 발생하고, “MLP 3번이 28% 기여했다”는 식의 설명은 비즈니스적으로 무의미했다. 비용과 설명 가능성 모두에서 문제가 있었다.

이 과정에서 핵심적인 리프레이밍이 발생했다: 전문가들을 모델 **밖에서** 결합하는 것이 아니라, 단일 모델 **안에서** 결합하면 어떨까? 이것이 PLE의 이중 전문가 아키텍처로의 전환 계기였다.

태스크 그룹 설계에서도 시행착오가 있었다. 초기에는 **GMM 클러스터 서브헤드** 방식을 시도했다 — K 개 클러스터 $\times$ T 개 태스크 = $K \times T$ 복잡도 폭발이 발생했다. 이를 4개의 Financial DNA 태스크 그룹(상품 보유 확률, 다음 상품, 고객 가치, 이탈 위험)으로 전환하여 구조를 안정화했다.

AI 에이전트 조직화

Phase별 AI 도구 운용

프로젝트의 각 단계에서 서로 다른 AI 도구를 전략적으로 배치했다. 각 도구의 강점을 살려 역할을 분담하는 것이 핵심이었다.

Phase A: 아이디어이션 (Gemini)

초기 컨셉 탐색과 브레인스토밍에는 Gemini를 활용했다. 광범위한 지식 베이스를 바탕으로 아키텍처 후보군을 빠르게 스캔하고, 다양한 접근법의 장단점을 비교하는 데 효과적이었다. ALS 대체 옵션, Black-Litterman 탐색, 모델 앙상블 접근법 등 다양한 아키텍처 후보를 Gemini와의 대화를 통해 탐색했다.

가장 큰 가치를 발휘한 것은 학제간 피쳐 아이디어의 탐색이었다. “화학 반응속도론으로 소비 행동을 설명할 수 있는가?”, “상품 채택이 전염병 확산과 구조적으로 동등한가?” 같은 질문을 던지며, 금융 고객 행동과 구조적으로 동형인 문제를 가진 학문 분야를 체계적으로 식별했다. PM/리드 아키텍트가 도메인 전문성(FRM, 신용 분석 경력)을 제공하고, Gemini가 학제간 연결고리를 제안하는 방식으로 협업이 이루어졌다. 이 과정에서 “구조적 동형사상”이라는 핵심 개념이 자연스럽게 부상했다.

Gemini의 광범위한 지식은 특정 기술의 깊이보다는 “어떤 분야에서 비슷한 문제를 이미 풀었는가”를 빠르게 탐색하는데 최적이었다. 11개 학문 분야에서 피처를 도입하겠다는 설계 방향이 이 단계에서 확립되었고, 이후 모든 기술적 결정의 근간이 되었다.

Phase B: 기술 검증 (Claude Opus)

아이디어이션 결과를 구체적인 아키텍처로 발전시키는 단계에서는 Claude Opus를 사용했다. 수학적 엄밀성이 필요한 loss function 설계, 데이터 리키지 검증, 정규화 파이프라인 설계 등 기술적 깊이가 요구되는 작업에 집중했다.

각 전문가의 실현 가능성을 하나씩 검증했다: “HGCN이 MCC 계층 구조에서 작동하는가?”, “Mamba가 17개월 시퀀스에 충분히 효율적인가?” 같은 질문에 대해 Opus와 심층적인 기술 토론을 진행했다. PLE vs MMoE의 트레이드오프 분석, adaTT의 loss-level vs representation-level 전이 비교 등 아키텍처 수준의 깊은 분석이 이 단계에서 이루어졌다. 특히 동질적 MoE에서 발생하는 전문가 붕괴(expert collapse) 문제를 Opus와의 대화에서 먼저 식별한 후, NeurIPS 2024의 sigmoid gate 논문을 발견하는 계기가 되었다.

Opus는 가정을 도전하는 데도 효과적이었다. “Black-Litterman이 정말 적합한가?”라는 반론을 제기하여 PLE로의 전환을 촉진했다. 이 단계에서 19개의 기술 참조 문서(.typ 파일)가 Opus와의 공동 작업으로 작성되었으며, 이 문서들은 이후 구현 단계에서 각 에이전트가 참조하는 설계 명세서 역할을 했다.

Phase C: 코드 환경 정비 (Cursor)

GitHub 기반의 코드 환경 구성, 프로젝트 구조 설계, 초기 보일러플레이트 생성은 Cursor로 수행했다. IDE 통합 환경에서의 빠른 코드 네비게이션과 리팩토링이 강점이었다.

이 단계에서 6개의 초기 설계 문서(00-09 아키텍처 명세)가 작성되었고, 가장 중요한 산출물은 CLAUDE.md 가이드일의 수립이었다. Config-driven 원칙, 관심사 분리, 리키지 방지 규칙 등 이후 모든 AI 에이전트가 따라야 할 “헌법”이 코드 한 줄 작성되기 전에 확립되었다. 이것은 의도적인 순서였다 — 가이드일이 먼저이고, 에이전트 실행이 그 다음이다.

Phase D: 병렬 구현 (Claude Code — Opus/Sonnet)

본격적인 구현 단계에서는 각 팀원이 AI 에이전트의 “팀장” 역할을 맡았다. Claude Code 환경에서 Opus와 Sonnet을 병렬로 운용하여 서로 다른 모듈을 동시에 구현했다. 이 단계가 가장 집약적이었으며, 3명의 인간이 각각 AI 에이전트 팀을 이끌었다.

PM/리드 아키텍트의 AI 팀은 아키텍처 수준의 결정(PLE config, adaTT 태스크 그룹, 로짓 전이 설계)에는 Opus를, 빠른 코드 구현(generator, adapter, pipeline runner)에는 Sonnet을 배치했다. 이 과정에서 label leakage 3건 탐지, FP16 NaN 원인 4가지 진단, GPU 활용률 최적화 등 핵심적인 디버깅 세션이 이루어졌다. 엔지니어 1의 AI 팀은 데이터 수집 파이프라인, HIVE 병렬 쿼리 로직, 10개 generator(TDA, HMM, Mamba, Graph, GMM 등)의 피처 엔지니어링, 피처-비즈니스 역매핑 레지스트리를 담당했다. 엔지니어 2의 AI 팀은 모델 학습, 수학적 검증, 지식 종류(PLE에서 LGBM으로)를 담당했다.

각 팀이 병렬로 작업하면서도 일관성을 유지할 수 있었던 것은 Phase C에서 수립한 CLAUDE.md 가이드일과 인터페이스 계약 검증 프로세스 덕분이었다. 파일 A가 저장하는 키와 파일 B가 읽는 키가 일치하는지를 매 병렬 작업 후 반드시 검증했고, 이를 통해 통합 시 발생할 수 있는 인터페이스 불일치를 사전에 방지했다.

Phase E: 실험 + 논문 (Claude Code Extension)

ablation 실험 단계에서는 Claude Code를 실시간 모니터링 도구로 활용했다. ablation 진행 상황, GPU 활용률, 여러 탐지를 실시간으로 수행했다. 이 과정에서 PLE toggle 버그를 라이브 디버깅으로 발견했다 — use_ple=false 설정이 전문가 구성 자체를 변경하여 공정한 비교가 불가능했던 문제였다.

실험 대기 시간을 활용한 문헌 조사도 이루어졌다. PLE의 val_loss가 수렴하지 않는 현상을 관찰하던 중, Opus와의 대화를 통해 softmax gate의 경쟁적 특성이 이중 전문가 간 수렴을 방해한다는 가설을 세웠고, NeurIPS 2024의 sigmoid gate 논문을 찾아 이론적 근거를 확보했다. 실험 결과 분석과 논문 작성이 동시에 진행되는 방식이었다.

논문 작성 단계에서는 4편의 논문(영문/한국어), 22개의 기술 문서가 Claude와의 반복적 작업으로 생성되고 다듬어졌다. 개발 스토리 자체도 Claude와 함께 프로젝트 과정을 되돌아보며 작성되었다. 이 단계에서 AI는 단순한 텍스트 생성기가 아니라, 프로젝트의 의미를 함께 구성하는 사고 파트너 역할을 했다.

문서 생산 규모

AI 에이전트와의 협업은 코드 구현에 그치지 않았다. 온프레임 프로젝트에서만 260개 이상의 문서(설계서 28개, 기술참조서 19개, 코드 리뷰 16건, 보고서 95건, 가이드 5개)가 생산되었으며, 총 30MB 이상의 기술 문서가 작성되었다. 이 중 상당수는 AI와 공동 작성 또는 AI가 초안을 작성하고 인간이 검수하는 방식으로 제작되었다. 특히 “소네트 작업 검증 리포트”는 Opus가 Sonnet의 코드를 검증하는 AI 간 리뷰 프로세스를 보여주며, “Claude Code Opus용 500+ 항목 체크리스트”는 AI 에이전트에게 체계적 검수 업무를 위임하는 방법론을 보여준다.

메모리 बैं크와 가드레일 시스템

온프레임 프로젝트에서 확립된 AI 관리 체계는 AWS 프로젝트에도 그대로 이식되었다. memory-bank 시스템(8개 컨텍스트 파일: projectbrief, activeContext, progress, techContext, productContext, systemPatterns, tasks, style-guide)으로 세션 간 맥락을 유지하고, .claude/RULES.md로 코딩 규칙을 강제하며, .cursorsrules와 동기화하여 Cursor AI와 Claude Code가 동일한 가드레일을 따르도록 했다. 심지어 Claude, Codex, Vertex AI 세 플랫폼에서 자동화 실험 브랜치(exp/claude-auto-*, exp/codex-auto-*, exp/vertex-auto-*)를 운영하여 AI 도구 간 비교 실험도 수행했다.

왜 Claude Code여야 했는가

이 프로젝트의 복잡도에서 결정적이었던 것은 긴 맥락 유지(1M context), 세션 간 메모리 बैं크, 서브에이전트 병렬 실행이었다.

Label leakage 3건의 연쇄 추적은 며칠에 걸친 작업의 맥락을 유지해야만 가능했다. 첫 번째 leakage(has_nba 중 북 컬럼)를 수정한 후, 같은 세션에서 두 번째(ground truth glob 정렬), 세 번째(generator label 입력)를 발견할 수 있었던 것은 이전 수정의 맥락이 살아있었기 때문이다.

FP16 NaN 4개의 동시 진단(CGC entropy, OT Sinkhorn, Causal DAG, logits)은 모델 아키텍처 전체를 한 번에 조망하면서 각 expert의 수치 연산을 추적해야 했다. 이는 파일 하나씩 보는 방식으로는 불가능했다.

실험 대기 중 NeurIPS 2024 sigmoid 논문을 발견하고, 우리 실험의 PLE softmax 미수렴 관찰과 연결하여 sigmoid gate를 구현한 과정도 연속된 맥락 안에서 이루어졌다 — 실험 결과 분석 → 문헌 탐색 → 이론 연결 → 코드 구현이 하나의 흐름으로 진행되었다.

논문 4편과 기술 문서 22편의 일관성을 유지하면서 동시 수정하는 것도, 전체 문서 체계를 기억하고 있는 에이전트만이 가능한 작업이었다.

AI 협업에서 발견된 핵심 패턴

프로젝트 전 과정에서 AI 협업의 반복적 패턴이 드러났다. 이 패턴들은 의도적으로 설계된 것이 아니라, 실제 작업 과정에서 자연스럽게 발현된 것이다.

1. “AI는 HOW, 인간은 WHAT과 WHY를 결정한다”

: AI가 코드와 텍스트를 생성하지만, 아키텍처 결정은 인간이 내린다. 구조적 동형사상이라는 핵심 통찰은 인간-AI 대화에서 발현되었지만, 그것을 설계 원칙으로 채택한 것은 인간의 판단이었다.

2. “에이전트 전에 가드레일을 세운다”

: CLAUDE.md는 코드가 아니라 코드 이전에 작성되었다. 헌법이 먼저이고 입법이 나중인 것처럼, 가드레일이 먼저이고 에이전트 실행이 나중이다.

3. “이종 AI = 이종 전문가”

: 모델의 이종 전문가 설계가 개발 도구 선택에도 그대로 적용되었다. 각 AI 도구가 특화된 역할을 수행하면서, 단일 도구로는 달성할 수 없는 품질과 속도를 확보했다.

4. “메모리 뱅크로 연속성 확보”

: 세션 간 맥락 보존을 위한 영속 파일 시스템이 AI 에이전트의 가장 큰 약점인 “맥락 망실”을 극복하는 핵심 메커니즘이었다.

5. “AI로 빠르게 실패한다”

: leakage, FP16 NaN, ablation 필터 미작동 등 수동으로는 수일이 걸릴 버그를 AI 에이전트와 함께 수분 내에 탐지하고 수정했다. 빠른 실패가 빠른 학습으로 이어졌다.

6. “AI는 코드 머신이 아니라 사고 파트너”

: 구조적 동형사상이라는 통찰, sigmoid gate 가설, 전문가 붕괴 문제의 식별 등 프로젝트의 핵심적인 지적 돌파구가 인간-AI 대화에서 나왔다.

Phase	AI 도구	역할
A. 아이디어이션	Gemini	컨셉 탐색, 아키텍처 후보 스캔, 학제간 브레인스토밍
B. 기술 검증	Claude Opus	수학적 검증, loss 설계, 리키지 분석, 기술 문서 19편 공동 작성
C. 환경 정비	Cursor	GitHub 구조, CLAUDE.md 가드레일, 설계 문서 6편
D. 병렬 구현	Claude Code (Opus/Sonnet)	3인 x AI 팀 병렬 코딩, 디버깅, 10개 generator 구현
E. 실험 + 논문	Claude Code Extension	실시간 모니터링, 문헌 조사, 논문 4편 + 기술 문서 22편

품질 관리 전략

CLAUDE.md 가드레일

AI 에이전트가 생성하는 코드의 품질을 보장하기 위해, 프로젝트 루트에 CLAUDE.md 파일을 두어 가드레일을 설정했다. 이 파일은 AI 에이전트가 매 세션 시작 시 읽어들이는 시스템 지침이다.

CLAUDE.md 핵심 가드레일

- **Config-Driven 원칙:** 모든 파라미터는 YAML에서 읽는다. 하드코딩 금지.
- **관심사 분리:** Adapter는 데이터 변환만, Runner는 파이프라인만, train.py는 학습만.
- **데이터 리키지 방지:** Scaler는 train split에서만 fit. temporal split에 gap_days 필수.
- **코드 검수 4단계:** 컴파일 검증 → 인터페이스 계약 검증 → 하드코딩 스캔 → 관심사 분리 검증.

메모리 뱅크

AI 에이전트는 대화 세션이 끝나면 맥락을 잃는다. 이 문제를 해결하기 위해 “메모리 뱅크” 시스템을 도입했다. 세션 진행 상황, 설계 결정 사유, 피드백 이력을 구조화된 마크다운 파일로 관리하여, 새 세션이 시작될 때 AI가 이전 맥락을 빠르게 복원할 수 있게 했다.

인터페이스 계약 검증

병렬 에이전트가 서로 다른 모듈을 동시에 수정할 때 가장 큰 위험은 인터페이스 불일치다. 파일 A가 저장하는 키 이름과 파일 B가 읽는 키 이름이 달라지는 문제를 방지하기 위해, 병렬 작업 후에는 반드시 “인터페이스 계약 검증” 단계를 수행했다. cross-file 키/필드 매핑 테이블을 자동 생성하여 불일치를 사전에 탐지했다.

기술적 도전과 해결

개발 과정에서 마주친 20건 이상의 기술적 문제를 5개 범주로 정리한다. 단순한 디버깅 기록이 아니라, 각 범주가 데스크톱 GPU 환경에서의 대규모 멀티태스킹 학습이 요구하는 엔지니어링 역량의 단면을 보여준다.

데이터 무결성 (Data Integrity)

학습 데이터의 오염은 모델 성능을 무의미하게 만든다. 이 프로젝트에서는 레이블 리키지와 스키마 불일치가 반복적으로 발생했으며, 각각을 체계적으로 탐지하고 방어 체계를 구축했다.

Label Leakage 3건

모델 학습 초기에 비정상적으로 높은 성능(AUC=1.0)이 관측되어 3건의 리키지를 발견했다.

- **중복 컬럼:** has_nba_1이 레이블과 상관관계수 1.0으로 존재. EXCLUDE 처리로 해결.
- **파일 로딩 순서:** glob 알파벳 정렬로 ground_truth.parquet가 benchmark.parquet보다 먼저 로드됨. 하위 디렉토리 분리로 해결.
- **Generator 입력 오염:** GMM 등의 generator가 레이블 컬럼을 입력으로 사용. label_cols 자동 제외 로직 추가.

LeakageValidator를 학습 전 필수 단계로 추가하고, CLAUDE.md 가이드라인에 검증 규칙을 명시하여 재발을 방지했다.

apply_ablation Schema 미갱신

Ablation 필터가 텐서에서 피처를 제거하는 데 성공했지만, `feature_schema["columns"]`가 원래의 316개 컬럼을 유지하여 모델이 매번 동일한 316차원 입력을 받았다. `apply_ablation`에서 텐서 제거와 동시에 `columns`, `num_features`, `feature_group_ranges`를 함께 갱신하도록 수정하여 해결했다.

FeatureRouter 활성화 — Expert별 피처 서브셋 라우팅

마일스톤: FeatureRouter가 활성화되어 각 expert가 전체 316D 피처 중 자신에게 지정된 feature group만 입력으로 받게 됐다. Expert별 입력 차원: `deepfm=109D`, `temporal_ensemble=129D`, `hgcn=34D`, `perslay=32D`, `causal=103D`, `lightgcn=66D`, `optimal_transport=69D`. 모델 파라미터가 4.77M → 2.8M으로 감소했다 (feature_group_ranges 패치 후 최종 기준).

구현 과정에서 두 가지 버그가 발생했다:

- **Config scoping 오류:** FeatureRouter가 `feature_group_ranges`를 읽을 때 잘못된 config 레벨을 참조하여 라우팅이 전혀 작동하지 않는 문제. config 경로를 명시적으로 지정하여 해결했다.
- **shared_{i} 이름 매핑 오류:** CGCLayer가 shared expert를 `shared_0`, `shared_1` 형태로 등록하지만, FeatureRouter가 이를 feature group key와 매핑할 때 인덱스 불일치가 발생. expert 이름 매핑 테이블을 별도로 관리하도록 수정했다.

`feature_groups.yaml`의 `target_experts` 선언이 실제 런타임 라우팅을 결정하며, 코드에는 expert 이름이나 컬럼 명이 하드코딩되지 않는다.

adaTT 포팅 버그 5건 — 온프레임 코드 비교로 발견

배경: Structure ablation에서 adaTT가 일관적으로 성능을 하락시켰다 (sigmoid: -0.006, softmax: -0.021, no PLE: -0.004). adaTT의 설계 문제가 아니라 포팅 과정의 구현 오류였다.

원인 발견: 온프레임(gotomoon) 소스코드와 1:1 비교하여 5건의 버그를 발견했다.

1. **Gradient 추출 빈도:** AWS는 에포크 마지막 배치에서만 gradient를 추출 (1회/epoch). 온프레임은 10스텝마다 추출 (17회/epoch). `_is_epoch_end_step` 플래그 대신 `global_step % grad_interval`을 사용하도록 수정.
2. **Config 로드 경로:** pipeline.yaml의 root-level `adatt`: 섹션이 `model_config`이나 `label_schema`에서 읽히지 않았다. `config.get("adatt")` fallback을 추가.
3. **freeze_epoch 미전달:** AdaTTConfig 생성 시 `freeze_epoch`을 전달하지 않아 항상 None. transfer weight가 끝까지 불안정하게 적응.
4. **Loss 구조:** 온프레임은 uncertainty weighting을 먼저 적용 (loss scale 정규화) 후 adaTT transfer. AWS는 either/or로 구현되어 adaTT 활성화 시 uncertainty weighting이 꺼졌다. 13개 태스크의 loss scale이 제각각인 상태에서 transfer → 큰 loss가 지배.
5. **warmup_epochs: 0:** affinity matrix가 identity (측정 없음) 상태에서 즉시 transfer 시작. 의미 없는 loss 공유.

수정 결과: sigmoid_adatt AUC 0.5605 → 0.5746 (+0.014). 피크(Ep6)에서 0.5786으로 sigmoid baseline(0.5771)을 초과. 더 결정적으로, 이 다섯 버그가 한때 “13-task 규모에서 adaTT가 PLE를 -0.019 AUC 손상시킨다”고 해석했던 전체 delta를 설명한다 — 버그 수정 후 adaTT on vs off 차이는 -0.019에서 -0.001로 수렴했고, 이는 단일 시드 노이즈 범위다. 초기 해석은 알고리즘적 발견이 아니라 이식 잔해였다.

교훈: preflight 로그 ("`AdaTT config: warmup=X, freeze=X, source=X`")를 추가하여 config 적용 여부를 학습 시작 전에 검증하도록 했다. MLflow가 있었다면 이 삽질을 상당 부분 방지할 수 있었을 것이다.

우리 “adaTT”는 Li 2023이 아니었다 — Naming Drift 발견

배경: Paper 3 기획을 위해 `core/model/ple/adatt.py`의 docstring과 참고문헌 블록을 다시 읽었다. 온프레임에서 그대로 들고 온 이름을 당연하게 쓰고 있었지만, 이름과 구현이 같은 족보를 공유하는지 확인한 적은 없었다.

발견: docstring이 인용하는 논문은 Fifty et al. NeurIPS 2021 (Task Affinity Groupings, TAG) 과 Chen et al. ICML 2018 (GradNorm) 두 편뿐이었다. Li et al. KDD 2023 "AdaTT: Adaptive Task-to-Task Fusion Network" 은 어디에도 등장하지 않았다.

진단: 우리 구현은 $L_i^{\text{adaTT}} = L_i + \lambda \sum_{j \neq i} w(i \rightarrow j) L_j$ 형태로, $w(i \rightarrow j)$ 를 gradient cosine similarity의 EMA로 추정한다. 이는 loss-level transfer이다. 반면 Li 2023 AdaTT는 expert activation 위에 learned gating을 얹은 **representation-level fusion**이며, native expert residual을 유지한다. 이름만 같을 뿐 완전히 다른 알고리즘이다.

합의: Paper 1의 "adaTT" 서술은 사실상 TAG + GradNorm 하이브리드를 평가한 결과였지, Li 2023의 메커니즘을 평가한 것이 아니었다. 온프레임 코드에서 상속된 이름 하나가 해석 혼란을 통째로 만들어 냈다. 리뷰어가 Li 2023 기준으로 본문을 읽었다면 재현 실패로 오해할 여지가 충분했다.

교훈: 코드 네이밍은 인용 족보와 일치해야 한다. 어떤 논문에 "inspired by" 되었더라도, docstring에는 **재현한 부분**과 **이탈한 부분**을 따로 명시해야 한다. 이름 하나로 5년의 서로 다른 연구 계보를 뭉뚱그리면 안 된다.

Paper 1 v1.1 — Algorithmic Finding 인가, Implementation Artefact 인가

배경: 5건의 포팅 버그를 고치고 naming drift까지 정리한 뒤, struct_13_ple_sigmoid 와 struct_13_ple_sigmoid_adatt 두 시나리오를 10 epoch, single seed로 다시 돌렸다. 버그 수정 전후의 숫자가 같은 이야기를 하는지 확인이 필요했다.

결과: sigmoid + adaTT의 AUC가 0.6541 (버그 수정 이전, Paper 1 초판에서 -0.019 degradation으로 보고된 값) 에서 0.6717 (버그 수정 이후) 로 움직였다. adaTT on/off gap은 -0.001, 노이즈 범위 안이었다. 즉 "adaTT가 성능을 떨어뜨린다"는 초판의 주장은 더 이상 데이터로 지지되지 않았다.

조치: Paper 1 v1.1 correction 커밋을 냈다. Abstract와 Section 5.4의 서술을 "degrades performance" 에서 "has null effect at 13-task scale" 로 재작성했다. 표는 버그 수정 후 숫자로 교체하고, 캡션에 5건의 버그 목록을 공개했다. Finding 2 ("156 task-pair instability" 가설) 는 "원래의 귀인은 그럴듯했으나 버그가 수정된 뒤에는 데이터로 뒷받침되지 않는다" 로 고쳐 썼다.

교훈: "알고리즘이 안 먹힌다"로 보이는 negative result가 사실은 implementation artefact였다. 이런 순간의 책임 있는 대응은 더 깔끔한 서사를 사후 재조립하는 것이 아니라, 수정을 투명하게 공개하는 것이다. 초판의 귀인(TAG affinity가 156 task-pair scale에서 불안정해진다) 은 합리적인 가설이었으나, 버그가 고쳐진 뒤에는 그 가설을 지지할 증거가 남지 않았다 — 그 사실이 그대로 기록되어야 한다.

Li 2023 원본 adaTT 재현 — AdaTT-sp 실험 결과

배경: Paper 1 v1.1 정정 후 남은 질문은 이것이었다. "우리 loss-level 변종이 null이면, Li 2023의 원본 representation-level adaTT는 다를까?" 이 시점까지 두 알고리즘은 이름만 공유했을 뿐 어느 쪽도 서로의 조건에서 평가되지 않았다. Paper 3를 기획하면서 원본 메커니즘을 우리 이중 expert basket 위에 구현하기로 했다.

구현 (AdaTT-sp): Li 2023의 핵심 메커니즘은 per-task fusion unit = softmax-weighted sum over experts + learnable scalar가 가중한 **native expert residual**. CGC gate에 별도 층을 얹지 않고 fusion_type: "cgc" | "adatt_sp" 플래그로 분기하여, adatt_sp 시 gated weighted sum 뒤에 task의 own task-specific expert 평균을 residual로 더했다. native_residual_weight는 초깃값 1.0의 학습 가능 scalar. pipeline.yaml
adatt_sp.enabled: false 기본, HP 플래그 use_adatt_sp=true로만 활성화. 신규 코드 50줄, main 동작 변화 없음.

결과 (10 예폭, single seed): struct_13_adatt_sp AUC 0.6696, Best NDCG@3 0.6825. 베이스라인 struct_13_ple_sigmoid (CGC gate, AUC 0.6728) 대비 AUC 변화량은 -0.0032였다. 참고로 loss-level 변종은 -0.0011이었으니, 원본 representation-level이 오히려 더 큰 하락을 보였다 — 3배 차이. Paper 3 기획 시점의 가설("원본이 더 잘 작동할 수도 있다")은 이 데이터에서 반증됐다.

교훈: 이중 expert basket이 이미 충분히 강한 inductive bias를 제공하면, per-task fusion mechanism을 뭘 쓰든 (loss-level TAG+GradNorm이든, representation-level Li 2023 AdaTT-sp든) 모두 null-to-negative 영역에 머무른다. 두 메커니즘의 “승패”가 아니라, **이 스케일에서는 fusion augmentation 자체가 필요 없다**는 것이 더 정확한 결론이다. Paper 3의 primary contribution이 “어느 fusion이 최선인가”에서 “**fusion augmentation이 언제 의미 없어지는가**”로 재조정됐다.

M1 Residual Complement 실험 — “선택받지 못한 신호 회수”의 실패

배경: AdaTT-sp 실패 후 남은 가설은 프로젝트 초기의 원형 직관이었다. “PLE gate가 특정 expert를 선택하면, 나머지 expert들의 신호 중에서도 쓸만한 것이 있지 않을까? adaTT를 대체할 잔여 신호 추출 메커니즘이 필요하다.” 이 가설을 직접 테스트하기 위해 cross-task 혼합(loss-level adaTT)이나 per-task own-expert 재주입(AdaTT-sp)이 아닌, **intra-task complementary recovery (M1)**를 설계했다.

설계: CGCLayer에 세 번째 fusion_type = “residual_complement”를 추가했다. primary = gated weighted sum은 그대로 두고, complement = $(1 - \text{gate_weights})$ 를 clamp 후 정규화하여 얻은 보수 가중치로 all_outs 가 중합을 추가로 계산, 최종 출력 = $\text{gated} + w_r \cdot \text{residual}$ (w_r 는 residual_recovery_weight, 학습 가능 스칼라). pipeline.yaml 기본 off, HP 플래그 use_residual_recovery=true로만 활성화. 신규 코드 30줄, 기존 경로 변화 없음.

결과 (10 에폭, single seed): struct_13_residual_complement final AUC 0.6675, best AUC 0.6692 (epoch 1). CGC baseline (0.6728) 대비 $\Delta = -0.0053$ 으로, 세 fusion 변종 중 **가장 큰 하락**이었다. 더 결정적으로, 최고 성능이 epoch 1에서 나오고 이후 단조 하락했다 — learnable recovery weight가 학습될수록 오히려 성능이 나빠졌다. Random 초깃값이 학습된 값보다 덜 해로운 상태라는 의미다.

4-way 비교 (struct_13 벤치마크, 10 에폭, seed=42):

Fusion	기제	Final AUC	Best AUC (ep)	Δ vs CGC
CGC gate	선택 expert weighted sum	0.6728	0.6728 (ep10)	—
Loss-level adaTT	cross-task loss mixing	0.6717	0.6733 (ep2)	-0.0011
AdaTT-sp (Li 2023)	per-task own expert residual	0.6696	0.6714 (ep3)	-0.0032
M1 residual complement	(1-gate) 미선택 회수	0.6675	0.6692 (ep1)	-0.0053

Per-task 분석: aggregate Δ 는 noise 수준이지만 태스크별로 뜯어보면 세 그룹이 드러났다. Group A (gate entropy 낮음, 3 tasks: segment_prediction, top_mcc_shift, mcc_diversity_trend)는 모든 recovery 방식에 둔감했다 ($|\Delta| \leq 0.003$). Group B (gate entropy 높은 중 2 tasks: churn_signal, will_acquire_lending)에서 M1이 크게 하락했다 (-0.020, -0.009). 유일한 positive outlier는 next_mcc (50-class, base F1 0.01 거의 random): $\Delta = +0.005$ 로 세 방식 모두 개선. 나머지 8 tasks는 $|\Delta| \leq 0.005$ 의 noise 영역.

Gate entropy 상관분석: joint_full 체크포인트의 마지막 CGC 층에서 task별 gate entropy를 추출해 recovery Δ 와 Pearson 상관을 계산했다. M1: $r = -0.40$, AdaTT-sp: $r = -0.32$, loss-level adaTT: $r = -0.31$. 세 방식 모두 같은 방향성 (entropy 높을수록 recovery 해로움)이지만, $n=13$, $p \approx 0.18$ 로 통계적 유의성 없음. Gate entropy가 recovery benefit을 구조적으로 설명한다고 주장할 근거로는 부족하다. Churn_signal과 next_mcc라는 두 outlier는 entropy가 아닌 각각 label construction과 base rate 같은 task-specific 요인으로 설명하는 편이 더 자연스러웠다.

세 실패의 공통 실패 모드: adaTT (loss-level), AdaTT-sp (per-task native residual), M1 (complement) 모두 **gate 출력에서 파생된 residual을 primary와 동일 fusion 지점에 additive로 주입**한다는 구조를 공유한다. CGC gate가 이미 AUC 0.6728 수준으로 near-optimal이면, gate의 역이나 gate가 낮춘 expert를 강제로 복원하는 것은 noise 추가에 불과하다. 세 실험의 공통 결론은 “gate-derived residual은 회수 가치가 없다”이다.

교훈: 구조를 개선하려면 residual 정의 자체를 gate와 독립적으로 재정의해야 한다. 후속 Paper 3 설계 방향은 (a) primary prediction의 **error** 위에서 학습하는 boosting-style residual path, (b) task-agnostic 전역 aggregation을 primary와 **parallel** 로 배치, (c) prediction uncertainty에 conditioned된 self-regulating 2차 gate — 어느 쪽이든 gate 역을 residual로 쓰는 3개 실패 공식을 구조적으로 회피하는 설계여야 한다. 이 세 가지 중 하나를 다음 실험으로 선택한다.

ECEB (Error-Conditioned Expert Bank) MV 실험 — 진단이 틀렸음을 드러낸 실패

배경: M1 실패 후 세 recovery 실험(loss-level adaTT, AdaTT-sp, M1 complement)의 공통 failure mode를 “gate 출력에서 파생된 residual을 primary fusion 지점에 additive 주입” 으로 진단했다. 이 진단을 검증하기 위해, residual 정의를 gate 출력이 아닌 **gate entropy (uncertainty)** 로부터 도출하는 ECEB minimum-viable 설계를 구현했다. 실패 모드를 구조적으로 회피한다고 가정한 설계였다.

설계 (MV-ECEB): CGCLayer에 네 번째 fusion_type = "eceb" 추가. primary = gated weighted sum 유지. recovery 경로는 **task-agnostic consensus** (all_outs 전체 평균)로 gate와 독립적으로 계산. recovery 가중치는 per-task learnable scalar $\sigma(w_t)$ 와 per-sample gate entropy ratio $\frac{H(g_t)}{\log} N$ 의 곱. 최종 = gated + $\sigma(w_t) \cdot \text{entropy_ratio} \cdot \text{consensus}$. gate가 혼란할 때 (high entropy) recovery on, 확신할 때 (low entropy) recovery ≈ 0 . pipeline.yaml 기본 off, HP 플래그 use_eceb=true로만 활성화. 신규 코드 50줄, 기존 경로 변화 없음.

결과 (10 예폭, single seed): struct_13_eceb final AUC 0.6665, best AUC 0.6670 (epoch 4). CGC baseline 0.6728 대비 $\Delta = -0.0063$ 으로, **네 실험 중 최악**. M1과 달리 monotone decline 은 아니고 0.665 0.667 범위에서 진동했으나 baseline을 한 번도 넘지 못했다.

5-way 비교 (struct_13 벤치마크, 10 예폭, seed=42):

Fusion	Residual 정의	Fusion 지점	Final AUC	Δ vs CGC
CGC gate	—	—	0.6728	—
Loss-level adaTT	cross-task loss mixing	primary loss	0.6717	-0.0011
AdaTT-sp	own task expert mean	primary repr (additive)	0.6696	-0.0032
M1 complement	(1 - gate) complement	primary repr (additive)	0.6675	-0.0053
ECEB (MV)	uncertainty $\times$ consensus	primary repr (additive)	0.6665	-0.0063

진단 수정: ECEB는 residual 정의를 gate 출력과 독립시켰는데도 실패했다. 즉 이전 세 실험의 공통 failure mode를 “gate-derived residual” 이라고 진단한 것은 부분적으로 틀렸다. 더 정확한 진단은: **primary 출력이 이 벤치마크에서 이미 near-optimal이고, 어떤 정의의 residual을 같은 fusion 지점에 additive로 주입해도 구조적으로 noise가 된다.** residual 정의의 영리함(gate inverse $\rightarrow$ uncertainty-gated consensus)과 무관하게, 설계가 침습적일 수록 Δ 절댓값이 단조 증가했다.

Paper 3 방향 재조망하기: 네 실험 모두 “primary representation에 residual을 더한다”는 공통 구조를 공유했다. 이 구조를 벗어나는 유일한 남은 방향은 A (BRP, Boosting-Residual Path): residual expert를 **primary의 prediction error 위에서** 학습시켜, primary representation을 건드리지 않고 최종 prediction 단계에서만 결합한다. B (TALA)는 parallel path지만 결국 결합 지점이 같아서 ECEB가 사실상 그 variant를 검증한 셈이다. 다음 실험은 BRP로 확정.

BRP (Boosting-Residual Path) MV 실험 — 처음 나타난 질적 차이

배경: 다섯 번째 실험. 네 fusion 변종(loss-level adaTT, AdaTT-sp, M1 complement, ECEB MV)이 모두 “primary representation에 residual을 additive로 주입” 한다는 공통 구조로 실패했다. BRP는 이 구조를 명시적으로 탈출한다: residual expert는 **output 공간** 에서 residual logit을 산출하고, primary의 detached prediction error 위에서 boosting 방식으로 학습된다. Primary representation은 건드리지 않는다.

설계 (MV-BRP): ResidualExpertBank 모듈 추가 — 각 task마다 별도 MLP head (hidden=[128,64], dropout=0.1), 입력은 마지막 CGC 층의 shared_concat (448차원 = 7 shared experts × 64 hidden), 출력은 primary tower output dim (binary/regression=1, multiclass=N). 학습 discipline: residual_target = y - "activation"(primary.detach()), residual loss = MSE(residual_logit, residual_target), 가중치 0.1로 total loss에 가산. Primary tower는 primary_predictions 사전에 전달되어 원래 ground-truth loss 그대로 학습. 추론/평가 시 predictions = primary + sigmoid(lambda_t) dot "residual" 로 결합 (λ_t 는 per-task 학습 scalar, init = -2.0 → sigmoid 약 0.12, 초기에 residual 억제). pipeline.yaml 기본 off, HP 플래그 use_brp=true 로 활성화. 신규 코드 200줄, 기존 경로 변화 없음. 추가 파라미터 3.6M → 총 3.56M.

결과 (10 예폭, single seed): struct_13_brp final AUC 0.6650 ($\Delta = -0.0078$ vs CGC, 6개 중 최악), 그러나 F1_macro 0.2117 (**6개 중 최고**, +0.0115 vs CGC), NDCG@3 0.7039 (**6개 중 최고**, +0.0219 vs CGC), val_loss 20.87 (6개 중 최저). 트레이드오프는 monotone degradation이 아니다 — primary 지표(AUC)는 하락하지만 ranking/multiclass 지표는 유의미하게 개선됐다.

6-way 비교:

Fusion	Final AUC	F1 macro	NDCG@3	Δ AUC
CGC baseline	0.6728	0.2002	0.6820	—
Loss-level adaTT	0.6717	0.2013	0.6646	-0.0011
AdaTT-sp	0.6696	0.1998	0.6570	-0.0032
M1 complement	0.6675	0.1998	0.6611	-0.0053
ECEB (MV)	0.6665	0.1998	0.6549	-0.0063
BRP (MV)	0.6650	0.2117	0.7039	-0.0078

Per-task 트레이드오프: next_mcc (50-class, base F1 0.01 거의 random)에서 F1이 0.0100 → 0.0440 (+340% relative)으로 **극적으로 개선**. 반면 churn_signal (baseline AUC 0.6868, binary 중 최고)이 0.6512로 -0.036 하락하여 aggregate AUC 손실의 대부분을 설명한다. 이 한 task를 제외하면 AUC도 거의 baseline 수준. 다른 binary 5개는 각각 -0.001 ~ -0.010 의 작은 손실.

Mechanism 진단: residual bank가 shared_concat 를 입력받고 residual MSE의 gradient가 shared experts 까지 역전파된다 (residual target만 detach, 입력은 아님). Primary 성능이 천장 근처인 task에서는 shared experts가 primary-supporting 최적점에 이미 수렴했는데 추가 MSE pressure가 그 최적점에서 끌어냈다. Primary가 고전하는 task(next_mcc)에서는 residual이 실제 신호를 보충한다. 즉 BRP는 capacity를 easy task 에서 hard task로 **재분배** 하는 “task-balance regularizer” 로 작동한다. 기존 fusion 실험들의 monotone degradation과 구조적으로 다른 pattern이다.

평가: 여섯 실험 모두 aggregate AUC 개선은 이루지 못했다. 그러나 BRP는 처음으로 **metric-selective** 개선을 보였고, primary representation을 건드리지 않는 구조적 경로가 최소한 일부 지표에서는 유효하다는 증거를 남겼다. Paper 3 narrative는 “fusion augmentation은 AUC 개선 경로로는 막혀있다” 는 명제를 여섯 실험으로 고정하고, BRP를 trade-off case (하드 태스크 구원 vs. 쉬운 태스크 손실) 로 기록한다. 더 정교한 variant (shared_concat detach, per-task enabling, residual_loss_weight tuning)는 후속 실험 스코프.

환경 전환 노트: 여섯 실험 모두 로컬 RTX 4070에서 scenario당 ~ 33분 학습으로 진행했다. 다음 단계(variant 실험, multi-seed, cross-dataset)는 로컬 GPU 시간 예산을 초과하므로 SageMaker 제출 경로로 전환한다. 로컬에서의 아키텍처 디버깅은 충분히 끝났고, 후속은 확장 실험 단계다.

BRP-detached (MV+) — Gradient Entanglement 가설 검증과 Paper 3 Positive Finding

배경: MV-BRP가 질적 trade-off를 보였지만 aggregate AUC는 여전히 6개 중 최악이었다 ($\Delta = -0.0078$). Per-task 진단은 churn_signal (기존 binary AUC 최고 0.6868)이 BRP 하에서 0.6512로 -0.036 하락한 것이

aggregate 손실의 대부분을 설명했다. 제안된 mechanism은 “shared experts가 primary-supporting 최적점에 수렴한 상태에서, residual bank가 shared_concat 를 입력받고 residual MSE의 gradient가 shared experts까지 역전파되어 그 최적점을 오염시킨다” 였다. 이 가설을 단일 수정으로 직접 검증할 수 있다: residual bank 입력을 `shared_concat.detach()` 로 바꿔 shared experts의 gradient를 완전히 격리한다. SageMaker 이관 직전 이 마지막 한 variant만 로컬에서 추가 실행했다.

설계: BRPConfig 에 `detach_input: bool = False` 필드 추가, `pipeline.yaml` 기본 off, HP 플래그 `brp_detach_input=true` 로만 활성화. forward pass에서 `brp_input = shared_concat.detach() if self._brp_detach_input else shared_concat`. 구조 변경은 이 한 줄뿐이며 파라미터 수, 학습 스텝 수, primary pathway 모두 불변.

결과 (10 예목, seed=42, 재시작 한번 포함): `struct_13_brp_detached` final AUC **0.6721** (CGC 대비 $\Delta = -0.0007$, noise 수준, 실질적 동급). Best AUC **0.6736** @ ep8 (CGC 0.6728을 +0.0008 초과). F1 macro 0.2075 (CGC 대비 +0.0073), NDCG@3 0.6965 (+0.0145, ep7에서 0.7315 peak), MAE 0.9688 (약간 손실).

Per-task 직접 검증 — 가설 완전 확인:

task	type	CGC	BRP (MV)	BRP-detached	진단
churn_signal	binary	0.6868	0.6512	0.6852	복원 (-0.036 → -0.002)
will_acquire_lending	binary	0.6549	0.6453	0.6553	복원
will_acquire_deposits	binary	0.6534	0.6493	0.6536	복원
will_acquire_investments	binary	0.6754	0.6719	0.6764	복원
next_mcc (50-class)	f1	0.0100	0.0440	0.0356	유지 (+340% → +256%)
others (8 tasks)	—	—	within noise	within noise	변화 없음

해석: MV-BRP의 easy-task AUC 손실은 구현 **artefact (shared_concat gradient leak)** 였고 알고리즘적 trade-off가 아니었다. `.detach()` 한 줄로 churn_signal은 0.6852로 복원되었으며 (Δ 만 -0.002, noise 안), 동시에 next_mcc의 F1 rescue는 +256% relative로 유지되었다. 이는 residual bank의 자기 파라미터만으로도 hard-task 구원에 충분한 신호를 학습할 수 있음을 보여준다. Shared experts는 primary-supporting 역할을 그대로 유지하되, residual bank가 별도 경로로 하드 태스크를 보충한다.

7-way 최종 비교:

Fusion	Final AUC	F1 macro	NDCG@3	Δ AUC
CGC baseline	0.6728	0.2002	0.6820	—
Loss-level adaTT	0.6717	0.2013	0.6646	-0.0011
AdaTT-sp	0.6696	0.1998	0.6570	-0.0032
M1 complement	0.6675	0.1998	0.6611	-0.0053
ECEB (MV)	0.6665	0.1998	0.6549	-0.0063
BRP (MV)	0.6650	0.2117	0.7039	-0.0078
BRP-detached	0.6721	0.2075	0.6965	-0.0007 (tied)

Paper 3 narrative 전환: 이전 6-way 결과는 “heterogeneous expert PLE + CGC gate 위에서 6개 fusion augmentation 모두 aggregate AUC 개선 실패” 로 서술했다. BRP-detached 추가 후 narrative는 정밀해진다: “Representation-additive fusion (adaTT 계열 + ECEB) 은 monotone degradation으로 실패. **Output-**

space boosting + shared-expert gradient 격리 (BRP-detached) 만이 AUC 보존 + F1/NDCG 개선을 동시 달성.” 이제 Paper 3는 순수 negative result가 아니라 **mechanism diagnosis + 구조적 해법 제시** 의 스토리다. 일곱 실험 중 여섯은 어디를 건드리면 안 되는지(primary representation에 additive), 그 여섯을 참조해서 일곱 번째가 어떻게 positive 영역에 닿았는지(shared concat detach + output-space boosting)를 보여준다.

Robustness 제한: single seed, single dataset. multi-seed 및 cross-dataset 검증은 SageMaker 경로에서 진행한다.

NEAS (Neglected-Expert Auxiliary Supervision) — 첫 AUC-Positive Finding

배경: BRP-detached가 primary representation을 건드리지 않는 구조적 해법으로 AUC를 baseline까지 끌어올렸지만 $\Delta \approx 0$. Paper 3의 **AUC 자체의 positive** 주장을 위해서는 다른 축의 mechanism이 필요했다. 브레인스토밍 중 떠오른 아이디어 — “게이트가 softmax로 expert 선호를 뚜렷하게 하면 무시된 expert로부터의 신호를 역가중치로 학습에 쓰면 어떨까?” — 에서 출발. 세 해석 중 **load-balancing auxiliary supervision** (expert collapse 방지 regularizer) 방향을 선택했다.

설계 (NEAS): 마지막 CGC 층의 gate_weights 와 all_outs 를 실시간으로 저장, per-task 역-게이트 집계 $inv_agg_t = \sum \frac{1-g_t}{sum} \cdot all_outs$ 를 계산. 이 집계 위에 per-task 보조 MLP head (128-64-output) 를 얹어 primary target y 를 예측하도록 학습. 보조 loss는 primary loss 계산 방식 그대로(pos_weight, class_weight 재사용)하되 aux_weight=0.05로 가산. 추론 시에는 **완전히 사용 안 함** — 학습 regularizer. NEASConfig 추가, HP 플래그 use_neas, 신규 파라미터 0.17M.

결과 (10 에폭, seed=42): struct_13_neas final AUC **0.6739** (CGC 0.6728 대비 $\Delta = +0.0011$, 양수). 10 epoch 내내 **monotone 상승** (ep5에서 baseline 크로스, 이후 단조). F1 macro 0.2019 (+0.0017), NDCG@3 0.6896 (+0.0076), MAE 0.9605 (CGC 0.9601과 실질 동급). val_loss 25.73 — 8개 시나리오 중 최저. Best AUC 0.6739 @ ep10 (overfit 없음, 여전히 상승 추세).

Per-task 패턴 — 균등 상승: 11/13 task에서 CGC 이상. Binary 7개 중 6개 개선 (+0.0004 ~ +0.0029), churn_signal tied, top_mcc_shift tiny loss. Multiclass는 nba_primary +0.0056 (의미 있는 개선), segment/next_mcc는 거의 무변화. Regression 3개 중 2개 미세 개선.

BRP-detached와의 성격 차이:

특성	NEAS	BRP-detached
Δ AUC aggregate	+0.0011 (positive)	-0.0007 (tied)
기제	Training-time load balance regularizer	Output-space error boosting
효과 패턴	전 task 균등 소폭 개선	Hard task 1개 대박 (+256% next_mcc)
Trade-off	거의 없음	대부분 task small loss
추가 파라미터	0.17M (aux heads only)	0.36M (residual_bank)
Inference 오버헤드	0 (training-only)	있음 (residual expert forward)

NEAS + BRP-detached 조합 — Non-Additive

배경: NEAS와 BRP-detached가 서로 다른 축에서 positive이므로 조합 시 additive 효과를 기대할 수 있었다. 가설: NEAS의 uniform 소폭 개선 + BRP-detached의 next_mcc 대박이 한 model에서 동시 발생.

설계: struct_13_neas_brp_detached — use_neas=true + use_brp=true + brp_detach_input=true. 두 mechanism 모두 activate, 구조 변경 없음.

결과 (10 에폭): Final AUC **0.6722** ($\Delta = -0.0006$), Best AUC 0.6733 @ ep7. **NEAS 단독의 +0.0011 이득이 소실.** F1 0.2062, NDCG@3 0.6864 — 둘 다 BRP-detached 단독보다 낮음. Trajectory는 ep7 peak 후 ep8-10 oscillation (NEAS 단독의 monotone 상승 패턴 소실).

Per-task 분석 — Conflict 지점: 조합에서 살아남은 것은 BRP의 `next_mcc` 구원 (+0.0250, 단독 +0.0256과 거의 동일). NEAS의 **uniform binary 개선은 전부 사라짐**:

task	CGC	NEAS	BRP-d	조합
will_acquire_deposits	0.6534	+0.0029	+0.0002	—0.0005 (사라짐)
will_acquire_lending	0.6549	+0.0021	+0.0004	—0.0000 (사라짐)
will_acquire_payments	0.6923	+0.0009	—0.0017	—0.0000 (사라짐)
nba_primary	0.1876	+0.0056	—0.0035	—0.0071 (두 쪽 모두보다 악화)
next_mcc	0.0100	+0.0007	+0.0256	+0.0250 (BRP 유지)
top_mcc_shift	0.6312	—0.0002	—0.0011	—0.0031 (additive 악화)

Mechanism of conflict: NEAS의 aux loss는 shared experts를 “모든 expert가 모든 task에 쓸모 있도록” 균등화하는 방향으로 gradient를 보낸다. BRP-detached는 primary loss를 통해 shared experts를 “**primary-supporting 최적점**”에 유지시킨다. 두 방향이 shared experts에서 정면 충돌 — NEAS는 generalist를 요구하고 BRP는 specialist를 유지하려 한다. 결과로 shared experts가 두 방향 사이에서 진동하며, NEAS의 regularization 이득은 소실되고 BRP의 residual bank 기반 error 보정만 부분적으로 생존한다.

Paper 3 narrative의 최종 형태: 9-way 결과가 세 영역을 명확히 그린다:

영역	구조	Δ AUC
실패 (5개)	Representation-additive fusion (adaTT / AdaTT-sp / M1 / ECEB / loss-adaTT)	—0.001 ~ —0.008
Positive (BRP-detached)	Output-space boosting + gradient isolation	—0.0007 (tied, F1/NDCG 크게 상승)
Positive (NEAS)	Training-time load-balancing regularizer	+0.0011 (실제 positive)
Non-additive	NEAS + BRP-detached 조합	—0.0006 (NEAS 이득 소실)

두 positive 영역은 **서로 다른 문제를 해결** — NEAS는 expert collapse 방지, BRP-detached는 primary error 보정. 독립적 positive finding이지만 **stacking 불가**. 운영 관점에서는 목적별로 택일해야 한다:

- Aggregate AUC 개선 + 전 task 균등 상승 → NEAS
- Hard task (near-random) 구원 → BRP-detached

이 non-additive 결과 자체가 Paper 3의 부가적 교훈이다: “**서로 다른 축의 positive mechanism을 stack해도 gain이 합쳐지지 않는다**” — shared representation이 한정 자원이기 때문에.

로컬 실험의 자연스러운 종결: 9개 시나리오로 이 벤치마크에서의 fusion augmentation 구조적 boundary가 완성. Multi-seed robustness, cross-dataset 재현, 조합 conflict 해소 variants 탐색은 SageMaker 경로에서 진행.

Causal Expert $W=0$ 붕괴 진단과 수정 — DAG가 사실은 죽어있었다

배경: Axis 3 (causal expert 재해석) 탐색의 사전 준비로 기존 baseline checkpoint의 causal expert가 실제로 DAG를 학습했는지 확인하기로 했다. `get_causal_graph()`가 반환하는 $W \circ W$ 를 simple하게 statistics 측정하는 20초짜리 진단. 결과가 예상 밖이었다.

진단: 체크포인트 5개 (로컬 4개 + 온프레임 2개)에서 `extraction_layers.0.shared_experts.4.W`를 모두 측정한 결과 — **전부 W frobenius ≈ 0.0001** . 초기 랜덤 init 스케일 (`randn $times$ 0.01`, 기대값 frobenius ≈ 0.32)

보다도 작다. 즉 학습을 거치면서 오히려 0 에 더 가깝게 **수렴했음**. 비대각 성분 중 $|W| > 0.01$ 인 entry가 **0%**. 어떤 의미 있는 DAG 구조도 학습되지 않았음.

체크포인트	W frobenius	$ W > 0.01$ 비율
struct_13_ple_sigmoid (CGC baseline)	0.0001	0%
struct_13_residual_complement (M1)	0.0001	0%
struct_13_eceb (MV)	0.0003	0%
struct_13_brp_detached	0.0001	0%
온프레임 gotothemoon best.pt	0.0001	0%

온프레임에서도 같은 현상이라는 것은 이게 AWS 포팅 버그가 아니라 **두 프로젝트가 공유하는 원래 구현의 구조적 문제**임을 뜻한다. 1년 넘게 Paper 1/2 에서 “causal expert가 DAG를 학습한다”고 서술했지만 실제로는 W 가 죽어 있었다.

근본 원인 — 잔차가 DAG를 우회한다: Expert의 forward는

$$z_{\text{hat}} = z + zW^2$$

잔차 z 가 latent 정보를 그대로 다음 단계 (causal_encoder MLP) 로 전달한다. Task loss 입장에서 W 를 0이 아닌 값으로 밀어낼 **구조적 동기**가 없다. NOTEARS 비순환성 + sparsity 정규화는 W 를 dense에서 멀어지게 하고 sparse를 선호하므로, **전역 최적**이 정확히 $W = 0$. 거기에 안착하는 게 optimizer 입장에선 당연하다.

더 나쁜 건 gradient도 같이 죽는다: task loss와 (추후 추가한) NOTEARS 재구성 loss 양쪽 모두 gradient가 W 자신에 비례 $-\partial(zW^2)/\partial W \sim 2zW$. $W \approx 0$ 이면 gradient도 0. 즉 $W = 0$ 은 **saddle point** 이고, 작은 init 스케일에서는 optimizer가 빠져나올 gradient 크기가 없다.

수정 — 두 부분 모두 필요:

- NOTEARS 재구성 loss 추가.** Zheng et al. (NeurIPS 2018) 원 논문이 의존하는 신호 ($\|X - XW\|_F^2$) 를 compressed latent 공간 버전으로:

$$\mathcal{L}_{\text{recon}} = \text{mean}\left((z - zW^2)^2\right), \quad \lambda_{\text{recon}} = 0.5$$

get_dag_regularization() 에 세 번째 항으로 추가, forward에서 z 를 _last_z 로 캐시.

- init 스케일 rescale.** 기존 randn \$times\$ 0.01 은 너무 작아서 saddle에서 gradient가 정확히 0. randn \$times\$ 0.1 로 10배 키움 — W^2 엔트리 scale이 $10^{-4} \rightarrow 10^{-2}$, forward의 residual 경로를 방해하지 않으면서 gradient가 의미 있는 크기로 전파.

한쪽만으로는 부족하다. SageMaker 검증에서 “recon loss만 추가, init은 0.01 그대로” 구성이 10 epoch 학습 후에도 여전히 $W = 0$ 을 반환했다. init까지 0.1로 올린 두 번째 시도에서 처음으로 W 가 살아있었다.

검증 결과 (SageMaker teacher_full, 10 epoch, softmax gate): W frobenius **0.338**, $|W| > 0.01$ 비율 **7.3%** (paper 권장 5–15% 범위), $h(W) = 0$ (완벽한 DAG), self-loop 0. Top edges: var_23 -> var_13 ($W^2 = 0.019$), var_9 -> var_13 (0.009), var_15 -> var_11 (0.007). var_13 이 sink 허브 역할 — 잠재 DAG가 있으면 기대되는 패턴이다.

그러나 task metric 에는 변화 없음:

Run	AUC	F1	NDCG@3	MAE
Pre-patch (local softmax, $W \approx 0$)	0.6729	0.2009	0.6814	0.9598
Post-patch (SageMaker softmax, W 학습됨)	0.6719	0.2042	0.6875	0.9597
Δ	-0.001	+0.003	+0.006	0

모두 noise 범위 안쪽. 구조적 버그 해결과 aggregate 성능 변화는 별개였다.

해석:

- Causal expert는 지금까지 causal_encoder MLP 단독으로 예측에 기여했고, W 는 작은 parameter 였다. W 가 살아나도 z_{hat} 변화 폭이 $W^2 \sim 10^{-2}$ scale 이라서 downstream MLP가 쉽게 흡수. **DAG 는 현 아키텍처에서 장식품** 이다.
- Paper 1/2 의 “causal expert 가 DAG 를 학습한다” 주장은 patch 이후야 **사실** 이 됐다. 이전에는 `get_causal_graph()` 가 noise 수준의 행렬을 반환하고 있었다. 설명력 claim이 이 함수에 의존했다면 patch는 pre-condition 이다.

Axis 3 재해석 탐색의 사전 조건 역할: CEH (causal explainability head), CRCG (causal reason-code generator), CTGR (causal task-group router) 같은 후속 설계들은 모두 **학습된 DAG 가 있다** 는 전제 위에서 작동한다. Patch 이전에는 그 전제가 성립하지 않았다. 지금은 성립하지만, **DAG 가 존재 → prediction 경로에 연결 → 지표 개선** 이라는 세 단계 중 첫 단계만 해결된 상태다. 두 번째/세 번째 단계 (routing + benefit) 는 Axis 3 실험에서 확인될 것이고, 그 결과가 positive로 나올 때 이 patch 의 **가치** 가 증명된다. 현재로서는 “구조적으로 꼭 필요한 선행 조치” 수준으로 기록한다.

Axis 3 첫 실험 — CEH (Causal Explainability Head)

선택 근거: 5개 Axis 3 후보 중 CEH 를 먼저 간다. 이유는 **구조 변화가 가장 작다** — MLP head 하나 + MSE supervision 신호 하나. Task routing 변경 없음 (CTGR 와 대비), serving 경로 분기 없음 (CG 와 대비), Paper 2 쪽 reason generator 연동 불필요 (CRCG 와 대비). “Finding 8 이후 학습된 DAG 에서 의미 있는 per-prediction 출력을 뽑을 수 있는가” 라는 가장 기본 명제를 가장 깨끗한 조건에서 검증한다.

설계: Causal expert 의 64-dim output 을 작은 MLP ($64 \rightarrow 64 \rightarrow \text{input_dim}$) 로 받아 per-sample per-feature attribution 벡터 생성. 학습 target 은 causal expert 자체 출력의 $\text{gradient} \times \text{input saliency baseline}$:

$$\mathcal{L}_{\text{attr}} = \text{MSE}(\text{attribution_head}(\text{output}), \nabla_x \text{output} \cdot \sum() \odot x)$$

Gradient $\times$ input 계산은 `requires_grad=True` 로 clone 한 입력에 한번 더 forward 돌려서 획득 — 메인 forward graph 건드리지 않음. Causal expert 내부에서 $\approx 14\%$ 추가 compute (7개 shared expert 중 1개의 부분).

Primary prediction 경로 그대로 유지, CEH 는 학습 시 regularizer + 추론 시 side output.

`expert._last_attribution` 을 inference 때 꺼내 감사 로그 persist — HMAC 서명 포함 audit integration 은 Paper 2 v2 범위.

MV 실험 (SageMaker teacher_full + CEH, 10 epoch, softmax gate):

지표	Pre-patch ($W \approx 0$)	Post-patch (CEH 없음)	Post-patch + CEH
Primary AUC	0.6729	0.6719	0.6734
F1 macro	0.2009	0.2042	0.1994
NDCG@3	0.6814	0.6875	0.6842
MAE	0.9598	0.9597	0.9609
W Frobenius	0.0001	0.338	0.366
$ W > 0.01$ sparse edges	0%	7.3%	8.5%
Attribution head 학습	n/a	n/a	확인됨

Attribution head bias (초기 0) 가 layer 0 에서 0.08 ± 0.08 , layer 3 에서 0.03 ± 0.04 로 이동 — MSE alignment loss 가 실제 gradient 를 head 로 전달 중.

관찰 — CEH 가 DAG 학습을 조금 강화: W Frobenius 가 no-CEH 대비 $0.338 \rightarrow 0.366$, sparse edge 비율 $7.3\% \rightarrow 8.5\%$. Attribution head 의 gradient 가 causal expert 의 representation 을 거쳐 W 에도 추가 신호로 작용한 것으로 보임. Primary AUC 는 $+0.0015$ 로 noise 범위 안, 그러나 Post-patch (no-CEH) 의 -0.001 감소와 대비하면 CEH 가 오히려 primary 를 유지시키는 경향.

MV 가 검증하지 못한 것:

- **Attribution 품질:** head 가 $\text{gradient} \times \text{input target}$ 에 fit 한 것은 당연. 그 target 이 규제기관/상담원/고객에게 실제로 유용한 설명인지는 sample 감사 + 다른 attribution method (Integrated Gradients, DAG path traversal) 대비 비교 필요.
- **Audit log 통합 효용:** per-sample attribution 벡터가 생산됐지만 HMAC 서명 + append-only 감사 로그에 연결되지 않음. Paper 2 v2 scope.
- **지표 개선 정당성:** AUC $+0.0015$ 는 9-way 비교의 noise 대역 안. Significant improvement 로 주장하지 않음.

Paper 3 narrative 업데이트: Finding 8 (W 붕괴 patch) 이 “DAG 가 존재하게 만들었다” 였다면, Finding 9 (CEH MV) 는 “그 DAG 에서 per-prediction output 을 뽑아낼 수 있음을 최소 scope 로 확인” — 3단계 (존재 $\rightarrow$ routing $\rightarrow$ benefit) 의 2단계 진입. 3단계 (실제 benefit) 는 audit 통합 + cross-dataset + human eval 후에 기재.

CEH Attribution 품질 평가 — 전역 중요도 붕괴 발견

MV 이후 바로 따라붙은 질문: “head 가 학습은 했는데, 그 출력이 실제로 **per-sample 설명** 인가, 아니면 모든 샘플에 대해 거의 똑같은 전역 중요도 벡터인가?” Post-hoc 평가로 검증 — SageMaker teacher_ceh checkpoint 에서 causal expert 를 추출, validation 5,000 건에 대해 네 가지 지표를 측정.

측정	값	해석
Spearman (CEH vs. $\text{grad} \times \text{input}$)	평균 0.259, 중앙값 0.252	훈련 타겟에 약 중간 정도 적합
샘플간 / 샘플내 분산비	0.055	샘플 내 피처간 차이가 샘플 간 차이보다 훨씬 큼
상위 10개 피처 중복도 (샘플 간)	0.791	서로 다른 샘플이 상위 피처의 약 80% 공유
입력 노이즈 대비 안정성 ($\sigma = 0.05$)	Spearman 0.985	매우 안정적 — 전역 패턴이면 당연한 결과

표 1: CEH attribution 품질 평가 결과. 낮은 분산비 + 높은 중복도의 조합이 핵심 — head 가 per-sample 설명이 아니라 평탄화된 **전역 중요도 벡터**에 수렴했다는 증거.

피처 그룹별 귀인 분포도 같은 얘기를 한다. CEH 는 `txn_behavior` 에 32.1% 를 할당 ($\text{grad} \times \text{input}$ 은 44.7%), `product_holdings` 에 12.6% vs. 5.5%, `product_hierarchy` 에 11.3% vs. 3.8%. 학습된 분포가 **자기 훈련 타겟** 보다는도 평탄하다.

추정 메커니즘: 훈련 타겟 (causal-encoder 출력 합의 $\text{grad} \times \text{input}$) 자체에 큰 샘플-불변 성분이 있고, 64-hidden 단층 MLP 가 그 전역 성분을 낮은 loss 로 잡으면서 샘플 특이적 잔차는 거의 버린 것으로 보인다. Head 가 고장난 게 아니라 타겟이 너무 평탄해서 얇은 MLP 가 per-sample 학습으로 강제되지 않는 상황.

무엇이 확인 / 반증 되었는가:

- 확인: Finding 9 MV 가 검증한 인프라 (DAG $\rightarrow$ attribution 소비자) 는 정상.
- 반증: 현재 타겟 설계만으로는 규제기관이 쓸 수 있는 per-sample 설명 생성이 불가능. CEH 에 의존하는 후속 Axis 3 후보 (특히 CRCG) 는 타겟 재설계 없이 이 baseline 에 쌓으면 안 됨.

다음 반복 (v2): `ceh.target_mode: "demeaned"` 도입. $\text{grad} \times \text{input} - \text{batch mean}(\text{grad} \times \text{input})$ 을 supervision 으로 사용, head 가 전역 패턴 대신 **샘플별 편차** 만 학습하도록 강제. 코드 변경량 최소 (3줄 + config 플래그 1개). “타겟이 평탄해서 붕괴” 가설을 직접 검증. 성공 시 분산비가 올라가고 top-K 중복도가 떨어져야 함. 실패 시 2차 안 (primary task gradient 타겟, 헤드 확대) 로 이행.

v2 결과 — 가설 검증, 전역 붕괴 해소

SageMaker teacher_ceh_demeaned (10 epoch, 나머지 조건 v1 MV 와 동일) 을 돌리고 동일한 post-hoc 평가를 적용한 결과:

측정	Raw (v1)	Demeaned (v2)	변화
샘플간 / 샘플내 분산비	0.055	0.719	13 ×
Top-10 피쳐 중복도 (샘플 간)	0.791	0.281	65% 감소
노이즈 안정성 ($\sigma = 0.05$)	0.985	0.953	여전히 안정
Primary AUC (churn_signal)	0.6866	0.6870	noise 범위

표 2: CEH attribution 품질 v1 vs. v2. 두 discriminative-power 지표가 한 자릿수 이상 개선되면서 primary AUC 는 그대로 — head 가 전역 패턴 재생산이 아니라 per-sample 편차 학습으로 전환됐고, downstream 태스크 손해 없음.

가설 검증 완료: 붕괴는 **타겟 설계의 artefact** 였지 head 용량이나 아키텍처 한계 가 아니었다. 3줄 + config 플래그 하나의 최소 개입 으로 per-sample 변별력 회복.

피쳐 그룹별 분포도 크게 재균형. txn_behavior 는 32.1% → 18.3%, gmm_clustering 은 28.5% → 21.4% 로 빠지고, product_hierarchy 는 11.3% → 30.3%, product_holdings 는 12.6% → 21.3% 로 올라감. Demeaned head 는 전역 고분산 그룹 대신 **샘플별 변별 신호** 를 담은 그룹 (상품 분류 체계) 을 선호.

주의: raw grad × input 과의 Spearman 은 0.259 → 0.096 으로 떨어지는데, 이는 당연 — v2 는 demeaned 타겟을 학습하므로 raw 는 더 이상 훈련 타겟이 아님. 품질 평가는 타겟-독립 지표 (분산비, top-K 중복도, 안정성) 로 해야 함. 또한 per-sample 변별력 확보는 규제용 설명성을 **필요**로 할 뿐 **충분**하지 않다 — 도메인 검증, 사람 평가, audit log 통합은 여전히 미완.

남은 후보: primary task gradient 타겟 (causal encoder 합 대신 특정 task logit 의 gradient 사용, “이 고객의 churn score 가 왜 높은가?” 라는 downstream 질문 과 정렬). Demeaned 인프라는 유지 하고 gradient 대상만 교체.

Audit Log 통합 (Paper 2 v2) — CEH 을 규제 가능 인프라로 연결

CEH v2 가 만든 per-sample attribution 벡터는 단독으로는 학습 지표일 뿐. 규제 관점에서 의미 있는 인프라가 되려면 “**이 고객의 이 결정을 왜 내렸는가**” 라는 질문에 답변할 수 있어야 함 — GDPR Art. 22 의 meaningful explanation 요구 + EU AI Act Art. 13 의 transparency 의무. Model promotion 은 “어떤 모델을 썼는가” 만 추적 가능, 개별 prediction 의 근거는 별도 경로 필요.

Paper 2 의 기존 AuditLogger (HMAC-SHA256 서명 + SHA256 hash chain + S3 Object Lock WORM 저장) 에 log_attribution 메서드 추가. 한 prediction 마다 1개 audit 엔트리 생성:

```
entry = {
    timestamp, operation: "attribution:{model_id}",
    sample_id,      # customer / request UUID
    top_features,   # [{feature, weight}, ...] top-K for human review
    attribution_hash, # SHA256 of full float32 vector
    prev_hash, hmac # 기존 chain linkage 재사용
}
```

3단 설명성: top-K (사람이 읽는 요약) + full-vector hash (forensic replay 검증) + chain linkage (tamper evidence). 감사관이 원본 입력을 재실행해 같은 attribution 벡터를 얻고 hash 가 일치하면 기록이 진짜, 불일치하면 변조 증거. Hash chain 은 특정 기록만 선택적 삭제 시도도 탐지.

코드 변경 최소화: CausalExpert.get_last_attribution() public accessor 추가, PLEModel.get_ceh_attribution() 으로 expert 찾기 캡슐화, AuditLogger.log_attribution() 한 메서드. 5개 단위테스트 + end-to-end smoke test (HMAC 생성 + chain 검증 + 변조 탐지) 모두 통과.

지금 시점 완성도: CEH 인프라 (Paper 3 Finding 9 v2) + audit log 통합 (Paper 2 v2 추가) = per-prediction 설명성에 대한 최소 가능 증거 세트. 남은 것은 (a) 실제 서빙 경로 (core/serving/predict.py) 에서 호출 삽입, (b) 감사관 쿼리 UI, (c) 도메인 전문가 human eval. 모두 infra 단위 작업이며 Paper 3 의 후속 실험 스코프가 아님.

Axis 3 두 번째 실험 — CG (Causal Guardrail), 음성→양성 피벗

CEH 가 “왜” 라면 CG 는 “이 예측 믿을 만한가” — per-prediction 신뢰도 플래그. SR 11-7 “known limitations” + EU AI Act Art. 9 위험관리 관점에서 필수. 기존 causal expert 체크포인트에 추가 학습 없이 post-hoc 분석으로 진행.

v1 (W-recon) 실패: 첫 formulation 은 Finding 8 의 NOTEARS reconstruction 잔차를 그대로 가드레일 점수로 쓰는 것. $\|z - zW^2\|_2^2$ 가 낮으면 학습된 DAG 에 잘 맞는 입력, 높으면 OOD 라는 가설. 5,000 validation + 3종 합성 OOD (uniform random / column permute / extreme tail) 로 평가.

결과: 모든 분포가 1.0 주변에 압축됨 (in-dist median 0.9995, OOD medians 0.9998 0.9972). p95 threshold 에서 TPR 6.8% / 8.1% / 0% — FPR 5% 와 구분 불가, 사실상 랜덤.

원인: 학습된 W 가 너무 작아서 ($\|W\|_F \approx 0.36$ on 32×32 , 따라서 W^2 는 Frobenius $O(0.13)$) zW^2 가 z 를 거의 안 건드림. 잔차비가 입력 무관하게 ≈ 1.0 . Finding 8 의 “decorative DAG” 관찰이 CG 로 직접 전파된 셈.

v2 (z-space Mahalanobis) 성공: W 자체가 약하면 causal latent z 의 분포 구조만 쓰면 어떨까? In-distribution batch 로부터 per-dim μ, σ 를 뽑고 $\text{score} = \sum_j \left(\frac{z_{i,j} - \mu_j}{\sigma_j} \right)^2$.

Probe	median	p95	p99
In-distribution	23.6	62.8	268.1
Uniform random	749.5	1200.9	1458.5
Column-permuted	537.4	1076.0	1336.1
Extreme-tail	479.3	479.3	479.3

표 3: CG v2 z-space Mahalanobis 점수 분포. 모든 OOD median 이 in-dist p99 보다 큼 → 거의 완벽한 분리. p95 threshold (62.8) 에서 세 OOD 유형 모두 TPR 100%, FPR 5%. 이 한 formulation 전환으로 가드레일이 작동함.

CG 는 DAG 구조가 아니라 causal latent 로 성립. 이 결과가 CTGR / CRCG 설계에 중요한 방향 제시:

- 두 후보 모두 학습된 W 가 강한 신호를 담는다고 가정 → 현재 W 는 존재만 하고 사용 불가
- 따라서 CTGR / CRCG 를 바로 돌리면 CG v1 과 같은 이유로 실패 예상
- 해법: (a) W 를 증폭 (큰 init, 더 강한 recon lambda, DAG-routed residual), (b) 혹은 W 대신 latent 기반으로 재설계

비용: \$0 (SageMaker 재학습 없음, 기존 체크포인트 post-hoc 분석).

Finding 11 W-증폭 실험 — “decorative DAG” 는 학습 선택의 결과

Finding 10 결과가 두 가설을 낳았다. (a) W 가 작아서 CG v1 이 망한 것인지, (b) $\|z - zW^2\|$ formulation 자체가 구조적으로 한계인지. (a) 를 검증하기 위해 w_init_scale : 0.1 → 0.3, $recon_lambda$: 0.5 → 2.0 로 재학습 (teacher_ceh_w_amp, 10 epoch, 나머지 baseline 과 동일).

지표	Baseline	W-amp	배수
$\ W\ _F$	0.363	5.028	13.9 ×
Active edges ($ W > 0.01$)	8.5%	59.5%	7.0 ×
Max $ W_{ij} $	0.11	0.77	7.0 ×
Primary AUC	0.6870	0.6865	noise

표 4: W-증폭 훈련 결과. 증폭은 공짜 — W 가 한 자릿수 이상 커졌는데 primary metric 은 변화 없음. Finding 8 의 “decorative DAG” 는 **아키텍처 제약** 이 아니라 **학습 선택의 artefact** 이었음이 확인.

CG v1 재측정: 같은 체크포인트에 Finding 10 의 동일 eval 을 돌렸다.

Probe	Baseline TPR	W-amp TPR
Uniform random	6.8%	22.7%
Column-permuted	8.1%	18.6%
Extreme-tail	0.0%	0.0%

표 5: CG v1 OOD 검출률 (FPR 5% 고정). 증폭 후 두 probe 는 chance 에서 약한 신호로 올라오지만 extreme-tail 은 여전히 0% — W 크기가 아니라 formulation 자체의 한계.

Extreme-tail 0% 의 의미: 모든 샘플을 동일 벡터로 만들면 z 도 동일, 잔차도 동일 — per-sample 지표는 **분포 감각** 이 없다. W 크기 무관. CG v2 (Mahalanobis) 는 참조 분포로부터의 거리라서 이 경우도 100% 처리.

CG v2 영향 없음: W 증폭 여부와 무관하게 세 probe 모두 100% TPR 유지. 즉 z latent 은 W 와 독립적으로 discriminative 정보를 담음.

정리된 것:

1. W 는 학습 가능 — 적절한 init + recon 으로 한 자릿수 이상 증폭 가능, primary 비용 없음.
2. Finding 10 의 CG v1 실패는 **부분적으로** W 크기 문제 (증폭 시 15-20 TPR 점 회복), **부분적으로** formulation 한계.
3. Latent-space CG 는 여전히 지배적. W 기반 가드레일은 보완재, 대체재 아님.

남은 질문:

- 증폭된 W 가 CTGR / CRCG / CCP 를 **작동** 시키는가? Finding 11 은 그 후보들이 돌아갈 **전제 조건** (W 가 의미 있음) 을 확보했을 뿐, 직접 답은 아직.
- 더 공격적 증폭 (init 0.5, recon_lambda 5.0) 이 v1 을 쓸 만한 수준으로 올리는가 아니면 primary 를 깨는가? 현재 config (0.3 / 2.0) 는 최소 개입 버전.

업데이트된 권고: 남은 Axis 3 후보 (CTGR / CRCG / CCP) 는 **amplified W teacher 위에서** 돌리되, 각각 **latent-기반 대안** 과 나란히 평가. 모든 비교에서 latent 버전을 이겨야 할 baseline 으로 보고 W 버전이 자동 우위라고 가정하지 않는다.

비용: \$0.13 (10ep spot g4dn, 985s billable).

수치 안정성 (Numerical Stability)

Mixed precision 학습은 속도를 2배 높이지만, FP16/BFloat16의 좁은 표현 범위가 NaN 전파를 유발한다. 4건의 underflow와 2건의 변환 오류가 발생했다.

FP16 Underflow와 NaN 전파

Phase 2 AMP 학습 중 CGC 엔트로피, OT Sinkhorn, Causal DAG, logits 연산에서 FP16 underflow로 NaN이 전파되었다. 작은 gradient 값들이 FP16 범위 밖으로 떨어진 것이 원인이었다.

BFloat16 NumPy 변환 및 GradScaler

BFloat16 텐서를 `.numpy()`로 변환 시 NumPy가 해당 dtype을 지원하지 않아 모든 validation metrics 계산이 실패했다. 모든 텐서에 `.float()` 캐스트 후 `.numpy()` 호출 패턴을 적용했다. 또한 모든 배치가 NaN인 극단적 상황에서 `scaler.step()`이 “No inf checks were recorded” assertion을 발생시켰다. backward count가 0이면 step을 건너뛰는 방어 로직을 추가했다.

인프라 및 환경 (Infrastructure)

데스크톱 GPU 1대 환경에서는 드라이버 충돌, 백그라운드 프로세스, 네트워크 제약이 실험 설계 자체를 바꿀 수 있다.

Docker GPU Passthrough 및 좀비 컨테이너

Windows에서 Docker GPU passthrough가 CUDA 버전 불일치로 불안정하여 로컬 Python 환경으로 전환했다. 또한 종료되지 않은 좀비 컨테이너가 GPU 메모리를 점유하여 학습 속도가 1/3로 저하되는 문제도 발생했다.

torch CPU/CUDA 버전 충돌

SageMaker SDK v3(3.7.0) 설치 시 torch가 CPU 버전으로 교체되어 GPU 학습이 불가능해졌다. SageMaker v2(2.257.1)로 고정하여 해결했다. conda 환경에서도 CPU/CUDA 빌드 충돌이 반복되어 캐시 정리 후 CUDA 버전을 명시적으로 지정하여 재구성했다.

torch conda 캐시 복구 및 Ollama GPU 점유

관공서 네트워크에서 `download.pytorch.org`가 방화벽 차단(403)되어 conda 캐시의 기존 패키지를 수동 복사하여 환경을 복구했다. 별도로, Ollama 자동 시작이 VRAM 2GB를 점유하여 batch size 선택에 영향을 미쳤다. 12GB VRAM 환경에서 백그라운드 프로세스 하나가 실험 설계를 바꿀 수 있다.

VRAM Spillover 분석

batch 6144에서 전용 12GB + 공유 11GB = 23GB(시나리오당 10시간), batch 2048에서 전용 9GB + 공유 0.1GB = 9.1GB(시나리오당 2시간). 공유 GPU 메모리는 PCIe 경유로 전용 VRAM 대비 10~20배 느렸다. 이 정량 분석으로 최적 batch size를 결정했다.

Lambda 250MB zip 한계와 Container Image 전환

Paper 2 구축 과정에서 서버 의존성이 유기적으로 늘어났다. `lightgbm`(LGBM 학생 모델), `lancedb`(추천 케이스/진단 저장소), `duckdb`(VSS 검색), `pyyaml`이 차례로 추가되었고, layer 기반 번들이 Lambda의 250MB zip 한계를 초과했다.

세 차례의 시도가 실패했다. 첫째, layer를 병합하여 배포하자 `libgomp.so.1 missing` 런타임 오류가 발생했다 — `lightgbm`의 OpenMP 의존성이 Amazon Linux base에 없었다. 둘째, 로컬에서 휠을 미리 받아 `pip install --no-index --find-links`로 주입하는 경로는 Docker 네트워크 내부의 SSL 인증 오류로 실패했다. 셋째, Windows Docker의 `~/.docker/config.json`에 남아 있던 `credsStore: desktop` 항목이 ECR 로그인을 차단하여 제거 후에야 `docker push`가 성공했다. 마지막으로 기본 빌드가 OCI manifest를 생성하여 Lambda가 거부했고, `docker build --provenance=false`로 Docker V2 manifest를 강제한 후에야 이미지가 실행 가능해졌다.

결과적으로 Lambda Container Image(10GB 한계, Python 3.10 base)로 이관했다. 교훈: 서버 의존성이 조용히 누적되는 환경에서는 zip+layer 250MB 천장을 사전 점검해야 한다. Windows에서 Container Image 이관은 credential, SSL, manifest 호환성을 차례로 밝으며 하루 정도의 반복이 필요하다.

LanceDB 설계 오류 — 원본 피쳐 저장에서 추천 케이스 누적으로

Paper 2 초기 설계에서 LanceDB에 349차원 원본 피쳐 벡터 전체 × 100만 고객을 저장했다. 스냅샷당 1.4GB가 생성되어 주기 저장 시 저장소가 선형 증가했다.

“왜 모든 고객을 Lance에 담느냐?” 라는 직설적 이의 제기를 스스로에게 던지고 나서야 진단이 명확해졌다.

LanceDB의 가치는 **결과가 붙은 케이스를 시간에 따라 누적**하는 것이지, 인구 전체의 현재 피쳐 상태를 스냅샷으로 미러링하는 것이 아니다. 올바른 단위는 “추천 케이스 1건 = 추론 로그 1건”이었다.

재설계는 세 가지로 정리되었다. 첫째, recommendation_cases 테이블을 Lambda 호출마다 기록한다 — user_id, timestamp, 13개 태스크 확률, L1 reasons, FDTVS 점수. 둘째, DiagnosticCaseStore와 TemporalFactStore가 동일한 LanceDB 인스턴스를 공유하도록 통합하여 신규 DB 의존성을 추가하지 않았다. 셋째, 원본 피쳐 행렬 덤프는 제거했다 — 필요 시 원본 parquet에서 재계산하면 된다.

교훈: 벡터 DB 도입 시 질문은 “어떤 현재 상태를 복제할 것인가”가 아니라 “시간에 따라 무엇을 누적할 것인가”여야 한다. 전자는 부풀려진 캐시를 만들고, 후자는 감사 추적과 A/B 분석이 가능한 유용한 궤적을 남긴다.

파이프라인 엔지니어링 (Pipeline Engineering)

대규모 데이터 처리와 ablation 오케스트레이션에서 발생한 시스템 수준의 문제들이다.

pandas에서 DuckDB/cuDF 전환

941K 행의 pandas 처리에서 메모리가 급증했다. DuckDB(CPU columnar)와 cuDF(GPU)로 전환하여 메모리 효율성과 처리 속도를 동시에 개선했다.

NVIDIA Merlin 생태계 평가와 선택적 채택

초기에 NVIDIA Merlin 생태계(NVTabular, HugeCTR 등)를 풀스택 솔루션으로 도입하려 했다. 그러나 본 시스템의 7종 이종 전문가 구조(DeepFM, Temporal Ensemble, HGCN, PersLay, LightGCN, Causal, OT)가 Merlin의 정형화된 파이프라인과 맞지 않았다. Merlin은 단일 모델 학습에 최적화되어 있어, 전문가별로 서로 다른 입력 형식과 연산 그래프를 요구하는 이종 아키텍처를 수용하기 어려웠다. 최종적으로 Merlin에서는 DataLoader 컴포넌트만 채택하고, 데이터 전처리와 피쳐 엔지니어링에는 cuDF를 직접 활용하며, 서빙/배포에는 Triton Inference Server를 도입했다. 풀스택 프레임워크를 평가하되 실제로 맞는 컴포넌트만 취하는 실용적 엔지니어링 철학을 반영한 결정이었다.

Subprocess Pipe Deadlock

ablation 오케스트레이터가 subprocess.run(capture_output=True)로 시나리오를 실행했으나, 대량의 stdout이 파이프 버퍼(64KB)를 초과하여 교착 상태가 발생했다. stdout/stderr를 파일로 리다이렉트하여 해결했다.

Ground Truth 파일 오류 로드

glob 알파벳 정렬로 benchmark_ground_truth.parquet가 원본 데이터보다 먼저 로드되어 Phase 0가 정답 변수로 피쳐를 생성했다. ground truth 파일을 하위 디렉토리로 분리하여 해결했다.

Batch Size 불일치 및 bash JSON 이스케이프

pipeline.yaml의 batch_size=2048을 run_ablation_manual.sh가 6144로 오버라이드하여 VRAM spillover가 발생했다. 모든 설정을 config 단일 소스로 통일했다. bash 스크립트에서 JSON 파라미터의 이스케이프 처리 실패도 별도로 수정했다.

Calibration pickle 실패 — local class는 joblib에 직렬화되지 않는다

SageMaker g4dn.xlarge Spot에서 증류 Job을 실행했다. 파이프라인은 teacher soft-label 생성 → 7개 LGBM student 증류 학습 → 3개 LGBM 직접 학습 → calibration → fidelity 검증 → drift baseline → 모델 저장의 순서로 구성되어 있었다. 49분 동안 모든 증류와 calibration fitting이 정상 진행되었고, 마지막 단계인

joblib.dump(calib, calib_path)에서 크래시가 발생했다.

에러 메시지는 다음과 같았다: pickle.PicklingError: Can't pickle <class

'containers.distillation.calibration.calibrate_students.<locals>._LGBMProbWrapper'>.

원인은 명확했다. `_LGBMProbWrapper`(학습된 LGBM을 `CalibratedClassifierCV`가 쓸 수 있도록 감싼 `sklearn` 호환 wrapper)가 `calibrate_students()` 함수 내부에 local class로 정의되어 있었다. Python pickle은 import 가능한 module path가 없는 local class를 직렬화할 수 없다. wrapper는 fit 시점에는 문제없이 동작하지만, `joblib.dump` 시점에서야 이 제약이 수면 위로 드러난다.

수정은 한 줄이었다 — `_LGBMProbWrapper`를 `containers/distillation/calibration.py`의 모듈 레벨로 이동시켰다. 동일 Job 재실행 시 2352초(약 39분, Spot 중단 없음)에 `rc=0`으로 완료되었다.

교훈: `joblib`으로 dump되는 `sklearn` 호환 wrapper는 반드시 모듈 레벨에 정의해야 한다. Python 에러 메시지는 난해하고 `joblib.dump` 호출 시점에야 표면화되므로, 수십 분의 학습 결과가 한순간에 날아갈 수 있다. 값싼 방어책은 downstream에서 사용되는 모든 wrapper 타입을 pickle하는 단위 테스트를 두는 것이다 — 수 초 안에 같은 버그를 잡는다. 부수적 교훈으로, Spot 인스턴스에서는 Job 말미의 크래시 비용이 특히 크다. Job 종료 시점의 통합 신뢰보다 Job 시작 시점의 fail-fast 체크가 우선이어야 한다.

모델 아키텍처 발견 (Architecture Insights)

ablation 실험과 학습 과정에서 모델 구조에 대한 근본적 발견이 있었다.

PLE Toggle 버그와 Ablation 필터 미작동

`use_ple=false` 설정 시 7개 이종 전문가가 MLP 1개로 축소되어 공정한 비교가 불가능했다. expert basket은 유지하고 PLE layering만 비활성화하도록 수정했다. 또한 `feature_group_ranges`가 컬럼 단위로만 저장되어 ablation 필터의 그룹명 매칭이 실패, 24개 시나리오 전체에서 AUC가 동일(0.913)했다. 그룹 레벨 키를 추가하여 해결했다.

GPU 활용률 최적화

초기에는 batch size 512로 GPU 활용률이 낮았다. batch size 증가, DataLoader 튜닝(`num_workers`, `pin_memory`), Phase 0 텐서 사전 저장을 적용하여 학습 처리량을 개선했다. 다만 12GB VRAM 제약으로 batch size는 2048이 상한이었으며, 이를 초과하면 shared GPU memory로 spillover가 발생하여 오히려 성능이 저하되었다.

Softmax vs Sigmoid Gate 발견

PLE의 `val_loss`가 Phase 2에서 3.702로 고정되고, `shared_bottom`(1 MLP)이 `ple_only`(7 expert)보다 낮은 `val_loss`를 보이는 역전 현상이 관찰되었다. CGC softmax gate의 경쟁적 특성이 이종 전문가 간 수렴을 방해한 것이다. NeurIPS 2024 논문의 sigmoid gate 이론적 우위를 확인하고 구현을 진행했다. 이종 전문가 아키텍처에서 gate 함수 선택이 성능에 결정적 영향을 미친다는 교훈을 얻었다.

설계 철학의 배경

“설득의 대상은 항상 사람이다”

금융 추천 시스템의 최종 소비자는 알고리즘이 아니라 사람이다. 고객은 “왜 이 상품인가?”를 묻고, 영업점 직원은 추천 근거를 설명해야 하며, 규제기관은 모델의 의사결정 과정을 검증한다. 확률값 하나만으로는 이 세 그룹 중 누구도 설득할 수 없다. 따라서 모든 설계 결정의 기준은 “이유를 설명할 수 있는가?”였다.

2축 분해 프레임워크

아키텍처의 핵심 구조는 2축 분해에 기반한다: **Financial DNA**(이 고객은 누구인가?) × **Data Modality**(데이터의 형태는 무엇인가?). Financial DNA 축은 상품 보유 확률, 다음 상품, 고객 가치, 이탈 위험의 4개 태스크 그룹으로 구성된다. Data Modality 축은 상태/스냅샷/시계열/계층/아이템의 5가지 피처 유형에 대응하는 이중 전문가로 구성된다. 이 2축의 교차점이 전체 모델의 학습 구조를 정의한다.

전문가 붕괴 문제와 이중 전문가의 필요성

동질적 MLP 전문가(예: 모두 같은 구조의 3-layer MLP)를 사용하면 학습 과정에서 **전문가 붕괴(expert collapse)**가 발생한다 — 게이팅 네트워크가 하나의 전문가만 선택하고 나머지는 사실상 사용하지 않는 현상이다. Pinterest와 Kuaishou의 대규모 실험에서도 이 문제가 확인되었다. 구조적으로 서로 다른 이중 전문가(LightGCN, Causal OT, TDA, GMM 등)는 입력 공간과 연산 방식이 다르므로 동일한 표현으로 수렴할 수 없어, 붕괴를 구조적으로 방지한다.

피처의 이중 역할: 예측 재료이자 설명 어휘

피처는 예측 성능을 위한 입력일 뿐 아니라, 추천 근거를 설명하는 어휘(vocabulary)이기도 하다. AUC 기여가 미미한 피처라도 영업점에서 “이 고객의 소비 엔트로피가 높아 다양한 상품 경험이 있다”와 같은 설명을 가능하게 한다면, 설명 어휘로서 대체 불가능한 가치를 가진다. 이 때문에 순수한 예측 성능 기준만으로 피처를 제거하지 않았다.

경제학에서 데이터 사이언스까지

이 프로젝트의 설계 철학은 PM/리드 아키텍트의 학문적 여정에서 비롯되었다. 경제학을 전공하며 의사결정 과학(Decision Science)을 배웠고, 금융공학을 거쳐 데이터 사이언스에 이르렀다. 이 과정에서 점점 근본적인 의문이 생겼다: “데이터 기반 방법론에서 과학은 과연 어디에 있는가?”

경제학은 수백 년의 과학적 방법론 — 가설 수립, 이론적 프레임워크, 반증 가능성 — 을 축적해왔다. 그러나 경제학 자체도 다른 학문에서 도구를 빌려온 역사가 있다: 일반균형이론은 물리학의 열역학 평형에서, 게임이론은 수학의 조합론에서, 계량경제학은 통계학에서 출발했다. 경제학이 과학인 것이 아니라, **학문 간 도구 전이가 과학 발전의 보편적 패턴**이다. 실제로 경제학 노벨상 수상자 중 상당수가 물리학·수학 출신이다 — Samuelson(열역학→경제균형), Black-Scholes(열전도→옵션가격), Nash(고정점정리→게임이론), Mandelbrot(프랙탈→금융변동성). 경제학의 가장 강력한 도구들이 다른 과학에서 온 것이다.

그러나 금융공학을 거쳐 데이터 사이언스로 오면서, 과학적 엄밀성은 점점 열어졌다. 머신러닝 모델은 수학적 구조의 납득이 가능했다 — 선형 회귀의 최소자승법, SVM의 마진 최대화, 트리의 정보 이득 등은 왜 작동하는지 설명할 수 있다.

하지만 딥러닝으로 넘어오면서 상황이 달라졌다. “신경망”이라고 하지만 실제 신경계의 구조를 깊이 연구한 것이 아니라 비유적 명칭에 가깝고, “왜 이 가중치가 이 값인가?”에 대한 답은 “데이터가 그렇게 학습시켰다”뿐이다. 설명할 수 있어야 과학이고, 과학철학이라는 학문까지 생겨서 반증가능성, 패러다임 전환 등을 논했는데, 현재의 딥러닝 접근법은 과학이라기보다 엔지니어링에 가깝다고 느꼈다.

구조적 동형사상 — 과학을 다시 가져오기

이 의문에 대한 답이 구조적 동형사상이었다.

이미 인류가 각 학문 분야에서 수백 년에 걸쳐 발견한 과학적 방법론들이 있다. 화학 반응속도론, 역학, 정보이론, 위상 수학 — 각각이 주어진 현상에서 어떤 시사점과 인과관계를 끌어낼 수 있는지, 지식의 최전선에서 노력한 결과물이다.

우리가 직면한 문제(금융 고객 행동 이해)의 구조를 제대로 인지했다면, 다른 학문이 이미 풀어놓은 구조적으로 동등한 문제의 해법을 가져올 수 있다. Shannon이 Boltzmann의 열역학 엔트로피를 정보이론에 가져온 것처럼, Black과 Scholes가 열전도 방정식을 옵션 가격에 가져온 것처럼.

이것이 11개 학문 분야의 피처를 도입하고, 각 분야의 수학적 도구에 특화된 이중 전문가를 설계한 근본 동기이다. 단순히 “피처를 많이 만들자”가 아니라, “어떤 과학적 질문을 던질 것인가”가 설계의 출발점이었다.

추천 시스템에서 과학적 방법론의 위치

이 프로젝트는 추천 시스템이 단순한 패턴 매칭을 넘어 과학적 이해에 기반할 수 있다는 가능성을 보여주는 시도이다. 모든 추천이 “비슷한 사람들이 이것을 샀다”(상관)에서 끝나는 것이 아니라, “이 고객의 소비 역학이 이러하므로 이 상품이 적합하다”(인과적 설명)로 갈 수 있다.

Pearl의 인과추론, Friedman의 항상소득가설, Boltzmann의 통계역학 — 이들은 각 분야에서 “왜?”에 답한 과학자들이다. 우리의 이중 전문가 아키텍처는 이들의 도구를 금융 추천이라는 맥락에 구조적 동형사상을 통해 가져온 것이며, 이를 통해 추천 시스템에 과학적 설명 가능성을 부여하려는 시도이다.

핵심 교훈

“AI가 코드를 쓰지만, 설계 판단은 사람이 한다”

AI 에이전트는 놀라운 속도로 코드를 생성하지만, 아키텍처 결정, 데이터 리키지 판단, 비용 최적화 전략은 사람의 도메인 지식과 경험에 의존한다. AI는 “어떻게(how)”에 강하지만, “왜(why)”와 “무엇을(what)”은 사람이 정의해야 한다.

“가장 위험한 순간은 AI가 ‘그럴듯한 코드’를 만들어낸 직후다.
그때 비판적 검토를 멈추면 기술 부채가 쌓인다.”

“가드레일 없는 AI 코딩은 기술 부채를 만든다”

CLAUDE.md 없이 AI에게 자유롭게 코딩을 시키면, 하드코딩이 늘어나고 관심사가 뒤섞이며 테스트 불가능한 구조가 만들어진다. 가드레일은 AI의 생산성을 제한하는 것이 아니라, 올바른 방향으로 유도하는 것이다.

“이중 전문가 철학이 개발 방법론에도 적용된다”

PLE 아키텍처의 핵심 철학인 “이중 전문가(Mixture of Experts)”가 개발 방법론 자체에도 적용되었다. Gemini는 넓은 탐색에, Opus는 깊은 분석에, Cursor는 빠른 환경 구성에, Claude Code는 구현에 각각 전문화되었다. 하나의 AI 도구가 모든 것을 하는 것보다, 각 도구의 강점에 맞는 역할을 배분하는 것이 더 효과적이었다.

성과

시스템 구축

추천 시스템

- 13-task 멀티태스크 학습
- 7-expert PLE 네트워크
- adaTT 태스크 간 적응적 전이
- Uncertainty weighting (Kendall et al.)
- 로짓 전이 3가지 방식

인프라 및 실험

- 24개 ablation 시나리오 (9 structure × 15 expert)
- SageMaker spot 인스턴스 활용
- Phase 0 (CPU) + Phase 1~2 (GPU) 분리
- Config-driven 파이프라인 아키텍처

문서화

프로젝트를 통해 생산된 기술 문서는 총 9편이다. 아키텍처 개요, 파이프라인 가이드, 전문가 상세, 피쳐 참조, PLE+adaTT 참조, Causal OT 참조, 증류 참조, 시간 참조, 규제 프레임워크가 모두 Typst 기반으로 작성되었다.

논문

논문 2편을 준비했다. Paper 1은 이중 전문가 PLE 아키텍처와 ablation 연구를, Paper 2는 추천사유 생성 파이프라인, 운영/감사 에이전트, 규제 준수를 다룬다. 두 논문 모두 프로젝트 레포지토리에 한국어/영어 버전으로 제공된다.

Ablation 분석에서 드러난 전문가 특화

24개 ablation 시나리오 (9 structure × 15 expert) 분석 결과, 태스크 유형별로 전문가 특화가 명확히 드러났다. LightGCN은 multiclass 태스크(다음 상품 예측)에서, Causal 전문가는 regression 태스크(고객 가치 추정)에서 가장 큰 기여를 보였다. 이는 이중 전문가 설계의 유효성을 실증적으로 확인한 결과다.

온프레임 운영 성과

온프레임 시스템은 규제 정합 관점에서 금융분야 AI 위험관리 프레임워크(AI RMF) 24개 항목 중 85%(11개 완전 + 9개 부분)에 대응하는 모듈을 자체 평가했으며, 미달 항목은 기술적 구현이 아닌 조직적 의사결정 사항이었다. 12개 규제 준수 모듈(AI 고지, 거부권, 인적 재처리, 공정성 모니터링, 이해충돌 방지, 쏠림 탐지, 프롬프트 인젝션 방어, 안전성 문서, 모델 카드, 감사 추적, 동의 관리, 품질 모니터링)이 구현되었다.

평가 지표 체계

태스크 유형별 gold standard 지표를 정립했다: Binary 분류는 AUC, Multiclass 분류는 Macro F1, Regression은 MAE를 기준으로 삼았다. 단일 지표로 모든 태스크를 비교하는 오류를 방지하고, 각 태스크의 특성에 맞는 엄밀한 평가를 수행했다.

운영/감사 에이전트 — “AI가 분석하고, 사람이 판단한다”

이 프로젝트에서 가장 야심찬 설계 도전은 추천사유 생성과 별개로, 파이프라인 전체를 자율 진단하는 에이전트 시스템을 구축하는 것이었다. 핵심 질문: “전담 MLOps 인력 없이 소규모 팀이 규제를 준수하면서 AI 시스템을 운영할 수 있는가?”

설계 과정: 대화에서 아키텍처로

설계는 단일 세션의 연속 대화에서 점진적으로 발전했다:

1. “에이전트를 넣자” → 운영(Ops)과 감사(Audit) 분리 → 비동기 분리 근거(레이턴시, 규제 독립성, 장애 격리)
 2. “모델 수준은?” → “대부분 룰 엔진이면 된다” → 결정론적 엔진 + LLM 대화 인터페이스 분리
 3. “온프레임은?” → “Bedrock 없이도 기본 기능 완결” → 온프레임 baseline + AWS Bedrock 확장
 4. “할루시네이션 리스크” → 3-에이전트 독립 투표 → “델파이는 수렴 편향” → 2-Round 하이브리드 (독립 투표 + 순차 심의) + 마이너리티 리포트 보존
 5. “케이스가 쌓이면 지식” → 진단 케이스 스토어(LanceDB) → 유사 검색 + 통계 + 대응 효과 추적
 6. “한국어 모델은?” → Claude Sonnet(AWS, L2a 리라이트) + Exaone 3.5(온프레임) → 태스크별 최적 모델 분리
- 매 단계마다 “이 설계의 약점은 뭔가?”를 반복하여 구멍을 메운 결과, 15개 섹션 3,800줄의 설계 문서와 21개 파일 ~4,800줄의 구현이 하나의 세션에서 완성되었다.

핵심 설계 결정

결정론적 엔진 위에 LLM을 얹는다

에이전트의 95%는 Python 룰 엔진(if-else, 임계값, 패턴 매칭)으로 동작한다. 48개 체크리스트 항목을 자동 판정하고, finding + likely_cause + suggested_action 형식으로 리포트를 생성한다. LLM은 “이 수치를 어떻게 해석할 것인가”를 담당자와 논의하는 데만 사용한다.

이 분리의 핵심 이유: LLM의 확률적 특성은 감사 에이전트에서 오히려 리스크가 된다 — 같은 입력에 다른 진단을 내릴 수 있다. 룰 엔진이 사실을 확정하고, LLM은 그 사실의 해석을 논의한다.

마이너리티 리포트: “놓치는 것이 오탐보다 위험하다”

3-에이전트 합의에서 소수 의견(1/3)을 구조적으로 보존한다. 처음에는 순수 델파이(순차 심의)를 고려했으나, “뒤로 갈수록 앞 의견에 끌려가서 소수 의견이 사라진다”는 문제를 발견. 최종 설계: Round 1에서 독립 투표로 마이너리티를 확정하고, Round 2에서 논거만 보강 — 마이너리티는 절대 삭제 불가.

이 이름은 영화 마이너리티 리포트에서 영감을 받았다: 3명의 프리콧 중 1명만 다른 예측을 하는 것.

합의 의미론: “PASS는 만장일치가 기본이다”

3-에이전트 합의 파이프라인을 Bedrock Claude Sonnet 4.6 × 3 병렬 호출로 구성하면서, 초기 ConsensusArbiter는 단순 다수결로 판정을 분류했다: 3명 중 2명이 PASS면 PASS, 1명이면 FAIL. 겉보기에는 상식적인 규칙이었다.

이 전체를 깨뜨린 건 다음 한마디였다. “2 PASS + 1 WARN, 이것도 마이너리티 리포트로 올려야지.” 다수결로 WARN을 삼켜버리면, 규제 보고 시점에 “왜 그때 경고가 묻혔는가?”에 답할 수 없다. 이견은 흡수되는 것이 아니라 **반드시 지표면으로 올라와야** 한다.

재설계는 세 줄로 요약된다:

1. PASS는 3/3 만장일치만 허용 — 단 한 건의 이견도 없어야 한다

2. 3명 중 누구라도 WARN 또는 FAIL을 내면, 전체 판정은 WARN으로 승격(escalation)
3. `minority_report` 필드는 모든 이견 에이전트의 추론을 **항상** 보존한다 — 삭제 불가

교훈: 규제 산업의 ML에서 합의 의미론은 **통계 문제가 아니라 컴플라이언스에 민감한 설계 결정**이다. SR 11-7 모델 리스크 관리 프레임워크가 기대하는 패턴은 “PASS는 만장일치, 이견은 무조건 에스컬레이션”이다. 다수결은 연구에서는 자연스럽지만, 감사 관점에서는 “소수 의견을 버린 근거”를 매번 정당화해야 한다. 디폴트는 예방적 임계값 (precautionary threshold) 쪽으로 기울어야 한다 — 놓치는 것이 오탐보다 위험하다는 원칙의 합의 레이어 버전이다.

구현 ≠ 배선: “만들었는데 호출하는 곳이 없다”

Paper 2 / Lambda 엔드투엔드 검증 과정에서 같은 패턴이 반복해서 드러났다 — 클래스는 완성되었는데, 런타임에 아무도 부르지 않는다.

- **Champion-Challenger 게이트**: `core/evaluation/model_competition.py`에 `ModelCompetition.evaluate()`가 완전 구현되어 있었지만, 테스트에서만 호출되고 있었다. `submit_pipeline.py`는 이를 우회하고 fidelity 통과 시 자동 승격시켰다.
- **ConsensusArbiter**: 구현은 끝났는데 초기 `OpsAgent` / `AuditAgent`는 호출하지 않았다.
- **DiagnosticCaseStore, TemporalFactStore**: 설계 문서와 코드에 클래스 정의까지 있었으나 Lambda가 한 번도 참조하지 않았다.
- **FDTVSScorer, SelfChecker**: 완성은 됐지만 `Lambda predict.py`에 배선되지 않았다.
- **recommendation_cases LanceDB 테이블**: 스키마만 정의되고 Lambda가 `append`를 수행하지 않았다.

트리거는 단순한 질문 두 줄이었다. “Paper 2 §5.10 했냐?” 와 “운영/감사 에이전트들도 랜스디비 참조하는 로직 구성되어있어?” 를 확인하기 위해 감사를 수행하자 **연결되지 않은 7개 피처**가 드러났다. 해결은 단일 커밋 feat:

Connect all 7 unlinked features to Lambda + agents.

교훈: **구현은 배선이 아니다**. AI 보조 개발은 모듈을 빠르게 찍어내지만, 통합(wiring) 단계를 자주 건너뛴다. 새 모듈이 생길 때마다 체크리스트 한 줄이 필요하다 — “누가 이걸 호출하는가, 그리고 엔드투엔드 테스트가 이 경로를 실제로 통과하는가?” 그리고 주기적으로 **모듈 인벤토리 ↔ 런타임 호출 그래프** 감사를 돌려야 한다. 생성 속도가 빠를수록 배선 감사 주기는 오히려 짧아져야 한다.

피처 역매핑 파이프라인 완성

추천사유 생성의 핵심 갭을 4단계로 수정했다:

1. 영문 fallback을 한국어 기본 템플릿으로 교체
2. 누락된 12개+ 피처 prefix-to-group 매핑 추가
3. `ReverseMapper`를 `InterpretationRegistry`의 Level RM fallback으로 통합
4. `generate_l1()`에 `InterpretationRegistry` 3-tuple enrichment 연결

이 4단계 수정으로 L1 사유에도 IG 방향 + 태스크 맥락이 반영된 한국어 해석이 포함된다.

구현 방법: 서브에이전트 병렬 구현

설계 문서 작성 → 구현 계획 수립 → Phase별 Sonnet 서브에이전트 병렬 실행 → 메인 에이전트(Opus)가 검수하는 패턴으로 구현했다. Pre-req(4건) → Phase 0(3건) → Phase 1+2(5건) → Phase 3+4(4건) → Phase 5(2건) → 미구현 보완(5건), 총 6라운드의 병렬 실행.

각 라운드마다 `py_compile` + `yaml.safe_load` + 인터페이스 계약 검증 + 하드코딩 스캔의 4단계 검수를 수행하여 “빠르게 만들고 꼼꼼히 검증”하는 리듬을 유지했다.

산출물

- **설계 문서:** 3,861줄(Typst) + 1,168줄(Markdown) + 온프레임 핸드오프 430줄
- **구현:** 21개 Python 파일, ~4,800줄
- **설정:** agent.yaml + agent_tools.yaml(38개 도구) + checklist.yaml(53개 항목)
- **문서 업데이트:** Paper 2 양쪽 + typst 10개 + design 6개 + guides 3개 = 총 20+ 파일
- **다이어그램:** Paper 2 플레이스홀더 3개 → fletcher 다이어그램, docs/typst ASCII 15개 → fletcher 변환
- **번역:** docs/typst/en tech_ref 5개 한→영 전문 번역

PaperClip 차용 (2026-04)

PaperClip (2026.3, GitHub 30K stars)의 “zero-human company” 철학은 우리 원칙과 충돌하지만, 3가지 운영 메커니즘은 차용할 가치가 있었다:

1. **Heartbeat 패턴:** 에이전트가 주기적으로 깨어나서 체크리스트를 실행하되, 변경이 없으면 HEARTBEAT_OK를 반환하고 다시 잠드는 구조. 우리 CP5(5분), CP6(1시간) 등 주기 점검에 적용.
2. **예산 캡 (선불 직불카드 모델):** 에이전트별 월간 토큰 한도, 80% 소프트 경고, 100% 하드 정지. 핵심은 **graceful degradation** — 예산 초과 시 LLM 호출만 차단되고 룰 엔진은 계속 동작한다. 이것이 우리의 “온프레임 baseline 이 LLM 없이 완결” 설계와 자연스럽게 연결된다.
3. **Full Tool Trace:** 모든 ToolRegistry.call() 호출을 자동 기록하여 “이 진단이 어떤 도구 호출을 거쳐 생성되었는가”를 완전 재현 가능하게 만든다.

LangMem의 **프롬프트 자기개선**은 의도적으로 차용하지 않았다 — 감사 관점에서 “누가 이 프롬프트를 승인했는가?”에 답할 수 없기 때문이다.

메모리 프레임워크 차용 (2026-04, 후속)

PaperClip 구현 직후, 또 다른 메모리 프레임워크들(Mem0, Zep/Graphiti, Letta, SuperLocalMemory)을 검토하였다. 여기서도 **프레임워크를 통째로 도입하지 않고** 4가지 패턴만 선택적으로 차용하였다.

Zep/Graphiti의 시간적 지식 그래프

“2026-03-15 시점에 고객 A의 상태는?” — 이 감사 질의에 답하려면 그 시점의 모델 버전, 피처, 판정이 모두 필요하다. 분산된 컴포넌트를 조인하기보다, (엔티티, 속성, 값, valid_from, valid_to) 스키마의 TemporalFactStore를 만들어 단일 필터로 해결했다. 기존 DiagnosticCaseStore와 같은 LanceDB 인스턴스를 공유.

SuperLocalMemory의 수학적 Decay

“3년 전 drift 해결 방식이 지금도 유효한가?” — 아니다. 최근 케이스가 더 관련성 높다. 하지만 오래된 케이스를 삭제하면 감사 요건(7년 보존) 위반이다. 해법: 원본은 보존하되 **검색 가중치만** $\exp(-\frac{\text{age}}{\tau})$ 로 조정. 반감기 90일 기본값. DiagnosticCaseStore.search_similar()에 30줄 추가로 해결.

Mem0의 팩트 압축

InterpretationRegistry가 피처 레벨 한국어 해석을 제공하지만, 고객 **서술적 프로파일**은 없었다. “이 고객은 적금 선호, 리스크 회피”라는 팩트가 L2a 프롬프트에 없으면 Claude Sonnet이 맥락 부족 상태로 사유를 쓴다. FactExtractor를 룰 기반으로 구현 — YAML config에 15개 규칙을 정의, Python eval() 샌드박스(__builtins__ 차단)로 안전하게 평가. **LLM 호출 0회**.

Letta의 Recall Memory

BedrockDialogSession이 세션 종료 시 대화 이력을 잃어버리던 문제를 DynamoDB 기반 DialogRecallMemory로 해결. 과거 대화를 임베딩 검색(또는 키워드 fallback)으로 조회하여 시스템 프롬프트에 주입한다.

품질 검수에서 발견된 연결 버그

구현 완료 후 인터페이스 계약 검수에서 **치명적 버그 1건**을 발견했다: `generate_ll()`에서 `customer_facts`를 retrieval했지만 `local` 변수에만 붙였고, SQS를 통한 L2a 경로에서는 전혀 전달되지 않고 있었다. 즉 M-3 체인이 **전 구간 no-op**이었다. `get_best_reason()`에서 `facts`를 재조회하여 SQS context에 직접 주입하는 방식으로 수정.

이 경험이 주는 교훈: **서브에이전트가 개별 파일을 잘 구현했더라도, 파일 간 데이터 흐름을 연결하는 것은 별개의 작업**이다. 메인 에이전트의 최종 검수가 필수적인 이유.

데이터 무결성 감사: v3에서 v4로 (2026-04-10/11)

“불균형 멀티태스크 테이블 데이터에서 AUC 0.98은 돌파구가 아니다 — 리키지 적신호다. 모델이 학습한 게 아니라, 입력으로 받은 버킷 함수를 역산하고 있을 뿐이다.”

결정론적 리키지 발견: 18개 → 13개 태스크

리키지 감사를 통해 모델이 실제로 무엇을 학습하고 있는지 추적한 결과, 4개 태스크를 제거했다. `income_tier`는 `income`의 직접 버킷이었고, `tenure_stage`는 `tenure_months`의 버킷, `spend_level`은 `synth_monthly_spend`의 버킷이었다. `engagement_score`는 $0.3 \cdot \text{is_active} + 0.4 \cdot \text{freq} + 0.3 \cdot \text{num_products}$ 라는 선형 결합 그 자체였다. 네 가지 모두 모델 입력 피처 또는 그 자명한 변환이었다.

감사는 검증 지표의 비현실적으로 높은 수치에서 즉각 촉발됐다: `income_tier` AUC = 0.98, `tenure_stage` F1 = 0.98. 불균형 데이터에 멀티태스크 테이블 설정에서 이 수준의 수치는 모델의 실력이 아니라 리키지 또는 자명한 복원의 명백한 신호다. 리키지 감사가 즉시 시작됐고, 원인은 레이블 정의 자체에 있었다: 4개 태스크 각각이 모델 입력에 이미 있는 피처의 결정론적 버킷 또는 선형 변환이었다. CLAUDE.md에는 이미 “입력 피처의 결정론적 변환으로 파생되는 레이블은 태스크로 사용하지 않는다”는 원칙이 적혀 있었다. 감사를 통해 이 원칙이 우리 자신의 태스크 목록에 적용되지 않았음이 확인됐다. 4개를 제거했고, 태스크 수는 18개에서 13개로 줄었다. 남은 13개 태스크는 진정으로 불확실한 예측을 요구한다: 상품 가입, 이탈, 다음 MCC, 지출 이동 등 — 모델이 고객 행동에 대해 실질적인 무언가를 학습해야 하는 태스크들이다.

더 넓은 교훈: 리키지 검토는 **레이블 정의**를 대상으로 해야 한다, 피처 파이프라인만이 아니라. 완벽하게 깨끗한 피처 파이프라인도 그 피처들의 함수인 레이블이 있으면 무너진다.

합성 데이터 반복: v2 → v3 → v4

합성 벤치마크에서 의미 있는 레이블 분포를 얻기까지 세 번의 반복이 필요했으며, 그 과정에서 우리가 검토하지 않았던 가정들이 드러났다.

v2는 균일 난수 MCC 할당과 고정 거래 금액을 사용했다. 결과는 MCC 의존 태스크에서 거의 무작위에 가까운 레이블이었다 — 데이터 생성기가 구조 없이 만든 결과를 모델이 예측하는 상황이었다. v3에서는 페르소나 가중 MCC(4 5× 선호 배율)와 30% 거래 고착성을 도입했다. 레이블은 개선됐지만 가입 태스크가 여전히 균일에 가까웠는데, 페르소나-상품 매핑이 너무 약해서 고객을 충분히 차별화하지 못했기 때문이었다.

v4에서는 세 가지 조정을 더 강하게 가져갔다: MCC 선호 배율을 8 12×으로 높여 페르소나별 지출 패턴을 뚜렷하게 만들었고, 고착성을 60%로 올렸으며, 전 상품에 걸쳐 가입률을 높이고, 행동 변화 허용 창(mode-shift window)을

넓혔다. 결과적으로 13개 태스크 전체에서 의미 있는 분포를 얻었다 — 학습할 무언가가 있을 만큼의 분산이 있으면서, 자명한 예측기가 지배하지 않는 수준이었다.

세 번의 반복에서 나타난 패턴: 각 수정이 **다음** 가정의 약점을 드러냈다. v2에서 MCC 무작위성을 수정하자 페르소나 매핑 문제가 보였다. 페르소나 매핑을 수정하자 가입률 문제가 보였다. 합성 데이터 설계는 본질적으로 반복적이며, 반복이 끝났다는 올바른 신호는 생성기 코드가 깔끔해 보일 때가 아니라 레이블 분포가 목표 도메인을 닮았을 때다.

HGCN vs. LightGCN 역할 혼동

두 그래프 전문가는 처음에 같은 종류의 데이터로 라우팅되어 있었다: 상품 공동 보유 관계. HGCN이 `product_hierarchy`(상품-고객 이분 그래프)를 받아서 LightGCN과 기능적으로 동일해졌다. 두 전문가 모두 협업 필터링 방식의 친화성을 학습했다. 온프레임 설계 의도 — HGCN은 **쌍곡 공간에서 트리 구조**를 학습하고 LightGCN은 **이분 친화성**을 학습한다 — 가 라우팅에 반영되지 않았던 것이다.

수정을 위해 `merchant_hierarchy` 생성기를 다시 작성하여 실제 MCC L1→L2 푸앵카레 임베딩(27차원)을 생성하도록 했다: 상인 카테고리 그룹과 하위 카테고리 간의 계층적 관계를 표현한다. `feature_groups.yaml`을 갱신하여 HGCN의 `target_experts`가 상인 계층 그룹을 가리키도록 하고, LightGCN은 상품 공동 보유를 유지하도록 했다. 이 변경 이후 두 전문가는 진정으로 다른 기능을 수행한다: HGCN은 쌍곡 공간에서 분류 거리를 인코딩하고, LightGCN은 구매 공동 발생 패턴을 협업 필터링으로 인코딩한다.

이것은 코드 버그가 아니라 설계 의도 보존 문제였다. 생성기는 유효한 임베딩을 만들었고, 라우팅이 잘못된 전문가에게 보냈다. Config 기반 라우팅(`feature_groups.yaml`)만이 이 문제를 지속 가능하게 관리할 수 있다 — 어댑터나 모델 코드에 하드코딩된 라우팅이었다면 잘못된 할당이 훨씬 더 오래 보이지 않았을 것이다.

인프라 및 도구 수정

이 세션에서 여러 인프라 문제가 발견되어 수정되었다.

전문가 라우팅 단위. 라우팅은 컬럼 수준이 아니라 피처 그룹 수준으로 지정해야 한다. 컬럼 수준 라우팅은 Phase 0 정규화 재정렬에서 살아남지 못한다: 로그 접미사 컬럼이 추가되면 컬럼 인덱스가 이동하고 라우팅이 조용히 깨진다. 피처 그룹 수준 라우팅은 `feature_groups.yaml`의 명명된 그룹을 사용하며 정규화 변환 전반에서 안정적이다.

피처 그룹 범위 인덱싱. `feature_group_ranges`는 그룹 내 전체 컬럼의 최솟값/최댓값 위치로 연속 블록을 구성해야 한다. 이전 로직은 `_log` 접미사 컬럼이 기본 컬럼 뒤에 추가될 때 깨졌다: 범위 끝점이 더 이상 올바른 인덱스를 포함하지 않았다.

태스크 유형별 지표 집계. 태스크 유형에 관계없이 AUC를 평균 내는 것은 오해를 낳는다. 올바른 집계는 이진 분류 태스크에 평균 AUC, 다중 클래스에 평균 F1-macro, 회귀에 평균 MAE를 사용한다. 유형 혼합 평균화는 개별 태스크 유형에 대한 모델 동작을 가리는 지표를 만들어냈다.

Ablation 결과 보존. 결과는 보관해야 하며 삭제해서는 안 된다. 정리 스크립트의 `rm -rf` 패턴을 아카이브 순환으로 대체했다: 새 결과를 쓰기 전에 각 실행의 결과가 타임스탬프가 붙은 하위 디렉터리로 이동된다. 이로써 완료된 ablation 실행의 우발적 손실을 방지한다.

Windows 슬립 및 서브프로세스 안정성. Windows의 절전 모드가 야간 실행을 중단시킨다. 장시간 세션을 위해 유휴 시 절전을 비활성화했다. 정상 실행에서 0이 아닌 종료 코드가 반환되는 산발적 서브프로세스 오류는 1초 백오프의 재시도 로직으로 해결했다.

LGBM 학생 모델의 온도 스케일링. 지식 종류 논문은 일반적으로 신경망 학생 모델에 $T = 3.5$ 를 권장한다. 트리 기반 LGBM 학생에는 $T = 10$ 이 적절함을 확인했다: LGBM은 높은 온도 소프트닝이 생성하는 부드러운 확률 분포를 표현할 수 없으므로, 신경망 학생 온도 설정은 오히려 해로웠다. $T = 1$ 은 교사의 순위 순서를 보존하면서 학생에게 불가능한 분포 형태를 강요하지 않는다.

모든 아키텍처 결정을 압도한 버그 (2026-04-13)

“불확실성 가중치 수정 하나로 NDCG@3이 +0.018, F1-macro가 +0.031 상승했다. 전체 ablation 실험에서 시도한 어떤 아키텍처 변경보다도 큰 개선이었다. 모델은 처음부터 설계대로 작동하고 있지 않았던 것이다.”

Uncertainty Weighting: 조용한 무작동

조사는 ablation 결과의 이상한 패턴에서 시작되었다. 아키텍처 복잡도가 높아질수록 — shared bottom에서 PLE로, transfer 없음에서 adaTT로 — NDCG@3이 개선되는 것이 아니라 오히려 하락했다. 근본 원인 조사를 촉발한 질문은 단순했다: 왜 더 정교한 아키텍처가 더 나쁜 추천 결과를 낼까?

답은 손실 계산 코드 안에 있었다. 온프레임 uncertainty weighting 구현은 불확실성 항 내부에 태스크별 `loss_weight`를 적용한다:

온프레임 (정상): $\text{loss} = \text{loss_weight} * (\text{precision} * \text{task_loss} + \text{log_var})$

AWS 포팅 (버그): $\text{loss} = \text{precision} * \text{task_loss} + \text{log_var} / 2$

AWS 포팅 과정에서 `loss_weight`가 완전히 누락되었고, 원래 수식에 없는 `log_var / 2` 나누기가 임의의 스케일링으로 추가되어 있었다. 결과적으로 `pipeline.yaml`에 선언된 `task_loss_weights`는 실험 초반부터 조용히 무시되어 왔다. 모든 학습 실행이 설정과 무관하게 13개 태스크를 동일한 가중치로 처리했다. 손실 규모가 큰 태스크가 그래디언트 업데이트를 지배했고, 시스템의 핵심 비즈니스 목표인 next-product 예측과 이탈 예측 태스크는 묻혀버렸다.

수정은 한 줄로 끝났다: `loss_weight` 곱셈을 복원하고 log-variance 항의 잘못된 /2를 제거했다. 결과: NDCG@3 +0.018, F1-macro +0.031. 이 개선 폭은 전체 ablation 연구에서 시도한 어떤 아키텍처 수정보다도 컸다.

이 교훈은 불편하지만 중요하다. 인프라 수준의 버그 — 모델 아키텍처가 아닌 학습 하네스의 버그 — 는 모델 버그보다 체계적으로 발견하기 어렵다. 피쳐 그룹을 잘못 라우팅하는 모델은 전문가 활성화 패턴이 불규칙하게 나타난다. 손실 가중치를 조용히 무시하는 학습 하네스는 정상적으로 학습하고, 수렴하고, 그럴듯한 메트릭을 출력한다 — 설정에서 지정한 목표를 전혀 최적화하지 않으면서.

Softmax-Sigmoid 역전

불확실성 수정 이전에 진행된 실험들은 sigmoid 게이팅이 일관되게 softmax를 앞섰다. 이 결과는 이질적인 전문가 간 경쟁적 softmax 정규화가 수렴을 방해한다고 주장한 NeurIPS 2024 sigmoid 게이트 논문과 일치했다. 이 발견은 결론으로 문서화되었고 이후 실험 설계에 반영되었다.

불확실성 수정 이후, 결과가 역전되었다: 이제는 softmax가 NDCG 메트릭에서 sigmoid를 앞선다.

되돌아보면 근본 원인은 깨진 손실 가중치로 추적된다. 13개 태스크가 모두 동일하게 가중될 때, 태스크 집합에서 수가 많은 binary 분류 태스크가 지속적으로 multiclass 및 regression 그래디언트를 압도했다. 이 조건에서 sigmoid 게이팅의 비경쟁적 특성은 진정으로 유익했다: 모든 전문가가 활성화 상태를 유지하고 binary 태스크 그래디언트에 잠식되는 것을 막을 수 있었다. softmax의 경쟁적 선택은 gradient-dominant binary 태스크에 전문가 용량을 집중시켜 문제를 악화시켰다.

올바른 손실 가중치 적용 후, multiclass 그래디언트가 의도한 강도를 회복했다. 그러자 softmax의 경쟁적 라우팅이 보호막으로 기능하게 된다: 전문가 전문화를 강제함으로써, binary와 multiclass 그래디언트 흐름 사이에 구조적 장벽을 만든다. 각 태스크 유형별 약한 전문가들이 지배적인 태스크에 의해 방향을 잃지 않는다.

동질적 MTL에서는 Sigmoid가, 이질적 MTL에서는 Softmax가 유리한 이유

동질적 MTL 문헌 (2 4개 태스크, 동일 태스크 유형)은 구조적으로 유사한 전문가 간 경쟁적 라우팅이 붕괴를 유발하기 때문에 sigmoid 게이트를 선호한다. 그러나 이 프로젝트의 13-task, 3-type 환경은 동질적 MTL이 아니다. Binary, multiclass, regression 그래디언트는 호환되지 않는 스케일과 업데이트 빈도를 가진다. 이 레짐에서 softmax의 경쟁적 전문가 선택은 라우팅 방화벽으로 작동하여, 태스크 유형별 전문가를 다른 태스크 유형의 그래디언트 오염으로부터 보호한다. sigmoid 문헌의 결과는 전이되지 않는다. 경계 조건이 중요하다.

이 에피소드는 더 넓은 방법론적 위험을 보여준다: 학습 버그가 최적화 과정을 오염시킬 때, 관찰된 결과는 잘못된 아키텍처 결론을 가리킬 수 있다. sigmoid 선호는 깨진 학습 환경에 대한 유효한 적응이었다. 환경이 수정되자, 근본적인 아키텍처 선호가 다시 드러났다. 근본 원인 조사 없이는 “sigmoid가 더 낫다”는 결론이 무기한 이어졌을 것이다.

adaTT at Scale: 13개 태스크, 156개 쌍 — 그리고 반복된 해석

초기 ablation에서 adaTT는 13개 태스크 구성에서 PLE 단독 대비 AUC를 -0.019 떨어뜨렸고, 이는 한동안 “adaTT가 13-task 규모에서 구조적으로 실패한다”는 서사의 근거였다. 156개 방향 쌍에서 친화도 추정이 불안정하다는 조합론적 설명, 그리고 PLE의 표현 수준 분리가 adaTT의 손실 수준 재혼합에 의해 무효화된다는 구조적 설명이 동시에 제시됐다. 이 해석은 이 개발 노트의 이전 판에 그대로 기록돼 있었다.

그러나 다음 세션에서 온프레임 코드와의 1:1 비교로 drift 5건이 드러났다 — gradient 추출 빈도(에폭당 1회 vs 10스텝당 1회), config 로더 경로(root-level adatt: 미워힘), freeze_epoch 미전달, uncertainty weighting과 adaTT의 순차 적용이 either/or로 구현된 점, warmup_epochs=0 기본값. 다섯 버그를 모두 수정하자 Sigmoid+adaTT AUC는 0.6541에서 0.6717로 올라갔고, adaTT on vs off delta는 -0.019 에서 -0.001 로 수렴했다 — 단일 시드 노이즈 범위 안이다.

즉 -0.019 는 알고리즘적 결론이 아니라 이식 과정의 구현 잔해였다. 13-task 규모에서 PLE + adaTT가 구조적으로 충돌한다는 이전 해석은 경험적 재검증을 통과하지 못했다. 현시점에서 adaTT의 이 규모 효과는 null — 의미 있는 이득도, 의미 있는 손상도 아니다. 친화도 추정 조합론과 표현/손실 수준 분해 불일치라는 가설은 여전히 이론적으로 그럴 듯하지만, 이 데이터셋·이 규모에서는 확인되지 않는다. 기록을 남기는 이유는 명확하다: 버그 수정 없이 아키텍처 결론을 내리면, 구현 잔해가 영구적인 “발견”으로 굳어진다.

GradSurgery: 실험한 그래디언트 수준 대안 (채택하지 않음)

PLE + adaTT 충돌의 진단이 새로운 실험을 동기화했다. 태스크 손실을 사후에 혼합하려는 시도 대신, 질문이 바뀌었다: 그래디언트 오염이 발생하는 지점 — 역전파 과정 — 에서 이를 방지할 수 있는가? GradSurgery를 구현하여 평가 하였으나, 최종적으로 PLE 단독 baseline 대비 의미 있는 개선이 없어 프로덕션에는 채택하지 않았다.

GradSurgery (Yu et al., 2020)는 충돌하는 태스크 그래디언트를 간접 그래디언트의 법선 평면에 투영함으로써, 다른 태스크의 성능을 저하시킬 성분을 제거하면서 도움이 되는 성분은 보존하는 방식으로 작동한다. 이 연산은 손실 수준이 아닌 그래디언트 수준에서 이루어진다.

PLE와의 아키텍처적 적합성은 직접적이다. PLE는 표현 수준에서 태스크를 분리한다. GradSurgery는 그래디언트 수준에서 그 분리를 보호한다. 156개 태스크 쌍 전체에 걸쳐 재혼합하는 대신, 그래디언트 투영이 태스크 유형 그룹 경계에 적용된다: binary 태스크를 하나의 그룹, multiclass 태스크를 또 다른 그룹, regression을 세 번째로. 이것은 이중 축 설계다 — PLE 라우팅을 위한 의미론적 그룹핑(Financial DNA 태스크 그룹)과 그래디언트 보호를 위한 기술적 그룹핑(태스크 유형).

이중 축 전문가 아키텍처 vs. 이중 축 학습 설계

모델 아키텍처는 2축 분해를 사용한다: Financial DNA (태스크 의미론) × 데이터 모달리티 (피처 유형). 학습 설계는 이를 반영한다: Financial DNA 그룹이 PLE 라우팅을 정의하고, 태스크 유형 그룹 (binary / multiclass / regression)이 GradSurgery 투영 경계를 정의한다. 이질적인 전문가를 만들어낸 동일한 구조적 통찰이 이제 이질적인 그래디언트 관리를 만들어낸다.

구현상의 도전은 계산적이다. GradSurgery는 쌍별 그래디언트 내적을 계산하기 위해 모든 태스크 역전파에 걸쳐 계산 그래프를 유지해야 하며, 그런 다음 그래디언트를 투영하고 재적용한다. 이 `retain_graph=True` 오버헤드는 12GB GPU에서 무시할 수 없는 수준이다.

12GB 카드에서의 VRAM: GradSurgery에서 얻은 교훈

GradSurgery 구현은 이 프로젝트에서 반복되는 주제를 다시 드러냈다: 모든 아키텍처 개선에는 VRAM 비용이 따르고, 12GB 데스크톱 GPU에서 그 비용은 언제나 가시적이다.

`retain_graph=True`는 모든 태스크 역전파에 걸쳐 전체 계산 그래프를 메모리에 유지한다. 13개 태스크에서 이것은 13개의 별도 그래디언트 테이프를 동시에 보유하는 것과 대략 동등하다. 기본 모델의 확립된 최적값인 배치 크기 2048에서, 이는 배치 크기 1024로의 감소를 필요로 하는 메모리 압박을 초래했다.

두 가지 완화 방법이 효과적임이 증명되었다. 첫째, `adaTT` 그래디언트 추출 빈도와 일치하는 `grad_interval=10` 스텝마다 그래디언트 투영을 수행하면 (매 스텝이 아닌) `retain_graph` 호출 횟수가 10배 줄어든다. 태스크 유형 간 친화도 관계는 천천히 변하기 때문에 투영 신호는 안정적으로 유지된다. 둘째, 156개 쌍 전체 대신 태스크 유형 그룹 간 투영을 수행하면 쌍별 내적 수가 $O(N^2)$ 에서 $O\left(G^2 \times \left(\frac{N}{G}\right)^2\right)$ 로 줄어든다 (G는 그룹 수) — 소규모 G에서 의미 있는 감소다.

별도의 VRAM 문제가 개발 환경 자체에서 발생했다. Ollama가 시스템 부팅 시 자동 시작하여 GPU 메모리에 언어 모델을 로드한다. 12GB 카드에서 이것은 약 2GB를 소비한다 — 배치 크기 결정이 9GB와 11GB의 활성 메모리 차이에 달려 있을 때 상당한 비율이다. 해결책은 Ollama 자동 시작을 비활성화하고 학습 실행 전에 프로세스를 종료하는 것이었다. 이것은 `LeakageValidator` 및 `preflight` 로깅과 함께 사전 학습 체크리스트의 표준 단계가 되었다.

이 프로젝트의 모든 VRAM 사고에 걸쳐 패턴은 일관적이다: 백그라운드 프로세스와 프레임워크 오버헤드는 학습 실행이 시작되어 메모리 압박이 이를 드러낼 때까지 보이지 않는다. 유일하게 신뢰할 수 있는 방어책은 사전 학습 VRAM 감사: 매 실행 전 `nvidia-smi` 확인, 사용 가능한 메모리가 확립된 기준선 아래이면 강제 중단.

GradSurgery 실험 결과: 미채택

이론적 적합성과 신중한 VRAM 완화에도 불구하고, GradSurgery는 ablation 평가에서 PLE 단독 baseline 대비 의미 있는 개선을 보이지 않았다. `retained computation graph` 오버헤드로 인해 배치 크기가 2048에서 1024로 감소했으나, 태스크 메트릭의 보상적 향상은 없었다. 따라서 최종 구성은 `adaTT`와 GradSurgery를 모두 비활성화하고, PLE의 전문가 라우팅이 강제하는 표현 수준의 분리에만 의존한다. GradSurgery 실험은 부정적 결과 증거로 여기에 기록된다: 그래디언트 수준 투영은 PLE의 전문가 라우팅으로 분리가 이미 강제될 때 아키텍처적 분리 대비 신뢰할 수 있는 개선을 제공하지 않는다.

금융 운영 체제와 태스크 결합: 가설과 현재까지의 증거

`adaTT`가 이 환경에서 실패한다는 관측이 유지되던 동안, 우리는 그 실패를 운영 체제 차이로 설명하는 가설을 발전시켰다. Meta 규모의 모집단 규모 데이터에서는 CTR과 CVR 같은 태스크 간 그래디언트 관계가 수십억 샘플 위에서 안정적이어서 `adaTT`의 손실 수준 전이가 실제 구조를 반영한다. 금융 기관의 1,000만 2,000만 고객 데이터는 표본 규모이고, 금리 변동·상품 정책 변경·규제 이동이 태스크 간 관계 자체를 역전시킬 수 있어 손실 수준 결합이 상관된 모델 실패(`correlated model failure`)를 만든다는 설명이었다. 재학습 주기(빅테크 시간 단위 vs 금융 주/월 단위) 차이가 이 결합 비용을 감당할 수 있는지를 결정한다는 것이 결론이었다.

이 가설은 매력적이고, 실무적으로도 유용하다. 그러나 그 근거가 되었던 -0.019 수치는 포팅 버그 5건 수정 후 -0.001 로 수렴했다. 따라서 “빅테크 방법론이 금융에서 역효과를 낸다”는 서사는 현재 이 데이터셋에서 실증적으로 뒷받침되지 않는다. 남는 것은 이론적 주장이다: 분포 이동이 잦고 표본 규모가 제한된 환경에서 손실 수준 태스크 결합은 격리된 아키텍처보다 수명 관리가 어렵다는 점 — 이는 여전히 금융 AI 배포의 설계 고려사항으로 유효하다.

기록 방침은 이렇다. 이 논의를 전체 삭제하지 않는 이유는, 모집단 vs 표본 체제 구분과 상관된 실패 모드에 관한 우려가 배포 결정에 영향을 주는 실제 리스크이기 때문이다. 그러나 이 우려가 adaTT 자체의 경험적 실패로 입증된 것처럼 과장해서는 안 된다. 다른 분포 이동 조건(실제 은행 데이터, 금리 충격 이후 재학습)에서 재검증이 필요한 열린 가설로 남겨둔다.

자기 조절 전문가: 침묵이 최선의 신호일 때

공동 어블레이션 분석은 예상치 못한 특정 원인을 드러냈다. 9개 전문가 유형 중 태스크별 지표 분해 결과, Causal 전문가 — NOTEARS로 구현된 — `segment_prediction` 태스크에서만 F1-macro -0.122 의 하락을 유발했다. 단일 전문가가 이 정도 규모의 손상을 일으킨 경우는 없었다.

조사를 통해 구조적 원인이 밝혀졌다. NOTEARS는 관측 데이터에서 방향성 비순환 그래프(DAG)를 복원하는 알고리즘이다. 데이터에 진정한 인과 구조가 존재하지 않아도 항상 DAG를 출력한다. 이 프로젝트에서 사용된 합성 데이터셋에서 레이블은 피처의 수식 기반 변환으로 파생된다. NOTEARS는 이 설정에서 통계적으로 일관된 엣지를 찾지만, 그 엣지는 실제 인과 관계가 아닌 수식을 반영한다. 알고리즘은 허위 의존성을 학습하고 이를 신뢰도 높은 밀집 표현으로 인코딩한다. Causal 전문가는 노이즈를 생성하는 것이 아니라, 완전한 신호 강도로 자신 있게 틀린 정보를 생성한다.

핵심 문제는 아키텍처적이다: Causal 전문가에게는 “모르겠습니다”라는 선택지가 없다. 현재 PLE 설계의 모든 전문가는 기여할 의미 있는 내용이 있든 없든 항상 출력 벡터를 생성한다. 전문가의 출력이 자신 있게 틀렸을 때, PLE 게이트는 그것을 무시하는 법을 학습해야 한다 — 하지만 게이트의 softmax 구조상 다른 전문가를 높이는 방식으로만 억제할 수 있고, 유해한 신호를 완전히 제거할 수는 없다.

해결책은 내부 신뢰도 게이트다. NOTEARS DAG의 총 엣지 가중치가 학습된 임계값 아래로 떨어지면, 전문가는 낮은 신뢰도 표현 대신 영벡터 — 침묵 — 을 출력한다. PLE 게이트는 그러면 오해를 유발하는 벡터 대신 아무것도 받지 않으므로, 그것을 무시하는 법을 학습할 필요가 없다.

더 넓은 함의는 아키텍처의 모든 전문가로 확장된다. 유용한 신호를 기여할 수 없을 때 스스로 침묵하는 자기 조절 전문가는, PLE 게이트가 어떤 전문가가 유해한지 파악해야 하는 부담을 없앤다. 부정적 전이의 위험 없이 새로운 전문가 유형을 자유롭게 추가할 수 있다 — 각 전문가가 언제 말하지 않아야 하는지에 대한 책임을 스스로 진다. 게이트의 역할은 “어떤 전문가가 유해한가?”에서 “이미 스스로의 관련성을 인정한 전문가들을 어떻게 가중치 부여할 것인가?”로 단순화된다.

자기 조절은 전문가 앙상블의 전제 조건

이 발견은 Paper 3의 핵심 질문과 직접적으로 연결된다: 의미론적으로 공약 불가능한 이종 전문가 출력을 어떻게 앙상블할 것인가. 자기 조절은 원칙에 입각한 앙상블의 전제 조건이다. 일부 전문가가 자신 있는 노이즈를 기여하면 앙상블 신호는 어떤 가중치 체계가 적용되기 전에 이미 오염된다. 내부 신뢰도가 낮을 때 침묵을 출력하는 자기 조절 전문가는, 게이트가 받는 모든 벡터가 강제적 기여가 아닌 진정한 관련성 주장임을 보장한다. 그러면 게이트는 상대적 유해성이 아닌 상대적 전문성을 기준으로 앙상블할 수 있다.

과적합, 게이트 엔트로피, 그리고 적응형 증류까지

30에포크 과적합: 손실-지표 분리 현상

30에포크 학습 실행에서 태스크별 지표 추적이 없었다면 놓치기 쉬웠을 발견이 나왔다. 훈련 손실은 15에포크 이후에도 단조적으로 하락했는데, 이는 정규화가 잘 된 모델에서 예상되는 동작이다. 동시에 분류 태스크의 검증 AUC는 plateau를 찍은 뒤 역전되었고, 회귀 MAE는 계속 개선되었다. 손실과 지표가 반대 방향으로 움직이고 있었던 것이다. 이 분리 현상에는 멀티태스크 학습 특유의 구조적 원인이 있다. 모델은 학습된 s_k 가중치로 13개 태스크 손실을 집계하는 불확실성 가중 복합 손실을 최적화하고 있었다. s_k 파라미터가 적응하면서 모델은 검증 성능이 저하 중인 분류 태스크에서 회귀 태스크로 실질적으로 가중치를 이동시켰다. 이 재가중치가 분류 성능 저하를 가려 복합 손실은 계속 하락했다. 단일 태스크 손실 곡선이었다면 plateau가 보였을 것이다. 멀티태스크 집계가 이를 숨겼다.

처방된 대응책 — 코사인 학습률 재시작 — 은 부차적인 문제를 낳았다. 각 재시작은 최적화 궤적에 모멘텀을 주입했고, 이로 인해 매끄러운 수렴 대신 손실 진동이 발생했다. 재시작 진폭이 단일 태스크 학습 예산에 맞게 설정되어 있어 13개 태스크 손실 공간에 비해 너무 컸다. 진동 자체는 개별적으로는 해롭지 않았지만, 체크포인트 선택을 위한 깨끗한 수렴 에포크를 찾는 것을 불가능하게 만들었다.

교훈: 태스크별 지표는 선택이 아닌 필수

복합 손실은 최적화에는 유용하지만 모니터링에는 기만적이다. 모델은 단조 감소하는 복합 손실을 달성하면서도 일부 태스크에서 조용히 성능이 저하될 수 있다. 태스크 유형별 분리된 검증 곡선 — 이진 태스크의 `avg_auc`, 다중클래스의 `avg_f1_macro`, 회귀의 `avg_mae` — 이 멀티태스크 수렴 감지를 위한 유일한 신뢰할 수 있는 신호다.

게이트 엔트로피: CGC는 분화되고 어텐션은 그렇지 않다

병렬 진단에서 학습 과정 전반의 게이트 가중치 분포를 조사했다. CGC 계층 게이트는 10에포크까지 의미 있는 분화를 보였다: 값이 0.33(단일 전문가 의존도 낮음)에서 0.88(거의 독점적 의존)까지 분포했다. 이는 예상되는 동작이다 — 서로 다른 태스크가 어떤 귀납적 편향이 어떤 예측 목표에 유용한지 발견하면서 서로 다른 전문가 선호를 발전시킨다. 태스크별 어텐션 계층은 반대 패턴을 보였다. 어텐션 가중치는 거의 균등한 분포(엔트로피 ≈ 1.0)로 수렴했고 학습 전반에 걸쳐 그 상태를 유지했다. 어텐션 메커니즘이 집종을 학습하지 못하고 평균화하고 있었다.

진단은 용량 불일치를 가리킨다. CGC 게이트는 7개의 이종 전문가 출력 — 서로 다른 통계적 특성을 가진 의미론적으로 구별된 벡터들 — 에 대해 작동한다. 전문가 출력이 진정으로 다르기 때문에 게이트는 강한 학습 신호를 얻는다. 태스크별 어텐션은 두 CGC 단계를 거쳐 서로 더 유사해진 표현에 대해 작동한다. 표현이 덜 분화되어 있으므로 어텐션 그래디언트가 약하다.

이 발견은 이후 실험의 아키텍처 결정에 영향을 미쳤다: CGC 게이트 가중치는 정보가 풍부하여 설명 가능성 신호로 직접 사용할 수 있지만, 태스크별 어텐션 가중치는 현재 아키텍처 규모에서 설명으로 신뢰할 수 없다.

3계층 풀백 아키텍처

PLE 교사에서 LGBM 학생으로의 지식 증류는 예상보다 어려웠다. 첫 번째 증류 시도는 표준 소프트 레이블 전이 (`temperature=5.0`, `alpha=0.3`)를 사용했고, 회귀 충실도는 적절하지만 소수 클래스 태스크의 분류 정렬이 부족한 학생을 만들었다.

실패 모드는 진단적이었다: 희귀 양성 클래스에 대한 교사의 소프트 레이블은 낮은 용량의 학생이 흡수할 수 없는 엔트로피를 포함했다. 학생은 다수 클래스 분포를 잘 학습하고 희귀 클래스 신호를 노이즈로 처리했다.

서로 다른 태스크 부분 집합에서 이런 실패가 세 번 발생하면서 단일 증류 구성이 13개 태스크 전체를 동시에 만족시킬 수 없다는 것이 확인되었다. 그 결과 설계된 것이 3계층 폴백 아키텍처다:

계층	모델	사용 시점	지연시간
L1	LGBM student (증류)	기본값: 전 태스크 총실도 > 임계값	~5ms
L2a	PLE teacher (직접 추론)	이진/다중클래스 태스크에서 LGBM 총실도 하락	~80ms
L2b	Bedrock LLM (Sonnet)	규제 설명 필요 또는 신뢰도 < floor	~800ms
L3	규칙 기반 폴백	모든 모델 불가, 회로 차단기 개방	<1ms

FallbackRouter는 롤링 윈도우에서 총실도 지표를 모니터링한다. 어느 태스크 유형에서든 LGBM 총실도가 설정 가능한 임계값 아래로 떨어지면, 라우터는 다음 계층으로 에스컬레이션한다. 교사(L2a)는 기본 서빙 경로가 아닌 신뢰성 안전망 역할을 하여 정상 상황에서는 학생의 지연시간 이점을 보존한다.

적응형 증류: 교사 임계값 게이팅과 Floor SKIP

표준 증류는 소프트 레이블을 무조건적으로 전이한다. 13개 태스크에 클래스 불균형이 심한 설정에서, 이는 학생이 희귀 클래스에 대한 낮은 신뢰도 교사 출력을 받고 이에 맞추려 한다는 것을 의미한다 — 교사 노이즈를 증폭하는 형태다.

적응형 증류는 두 가지 메커니즘을 추가한다. 첫째, 교사 임계값 게이팅: PLE 교사의 소프트 레이블은 교사 자체의 신뢰도(다중클래스 태스크의 경우 evidential 불확실성 $u_k = \frac{K}{\sum(\alpha)}$ 로 측정)가 태스크별 임계값을 초과할 때만 학생에게 전이된다. 임계값 이하에서는 학생이 해당 샘플-태스크 쌍에 대해 하드 레이블로 폴백한다. 둘째, Floor SKIP: 특정 태스크에서 교사의 신뢰도가 최소 floor 아래이면, 해당 태스크-샘플 쌍의 증류 손실을 0으로 설정한다 — 학생이 불확실한 교사에 동의하지 않아도 페널티를 받지 않는다.

결합 효과는 학생이 교사가 확인하는 출력에서 학습하고, 교사가 불확실한 태스크-샘플 쌍은 무시하는 것이다. 증류 신호가 교사 출력 분포의 고신뢰도, 고정밀도 부분에 집중된다.

3-에이전트 추천사유 파이프라인: FactExtractor → TemplateEngine → SelfChecker

추천 사유 생성 시스템이 단일 LLM 호출에서 3-에이전트 파이프라인으로 재구성되었으며, Bedrock(Claude Sonnet)을 통해 완전히 연결되어 있다:

- FactExtractor:** 원시 모델 출력 — 게이트 가중치, evidential 불확실성, IG attribution 상위 피쳐, 대조 쌍 — 을 수신하고 구조화된 사실 묶음을 생성한다. 이 단계에서는 자연어 생성이 없으며, 구조화된 추출과 검증만 수행한다.
- TemplateEngine:** 사실 묶음을 수신하고 적절한 레지스터(고객 대면, 행원 대면, 규제기관 대면)로 초안 사유를 생성한다. 템플릿 선택은 LLM의 추론이 아닌 요청 컨텍스트에 의해 구동된다.
- SelfChecker:** 초안 사유와 원본 사실 묶음을 수신하고 일관성을 검증한다. 사유의 모든 주장이 사실 묶음의 사실로 추적 가능한지 확인한다. 승인 또는 TemplateEngine으로 다시 라우팅되는 구체적인 불일치 보고서를 반환한다.

3-에이전트 구조는 단일 LLM 사유 생성에서 나타난 문제를 해결한다: 모델 내부 세부사항의 환각. “이 추천을 설명하라”는 프롬프트를 받은 단일 LLM은 그럴듯하지만 조작된 피쳐 기여를 생성하기도 했다.

FactExtractor→SelfChecker 루프는 검증된 모델 출력에 모든 사유 주장을 근거로 두어 이를 불가능하게 만든다. 추천사유 생성을 포함한 전체 서빙 파이프라인의 SageMaker end-to-end 비용은 현재 구성에서 사이클당 약 \$0.69이다.

향후 계획

학술 및 업계 발표

- 논문 완성: 2편 (Paper 1: 아키텍처 + ablation, Paper 2: 서빙 + 운영/감사 + 규제 준수)
- **DuckDB 커뮤니티**: pandas를 DuckDB로 대체한 ML 파이프라인 사례
- **Anthropic 케이스 스터디**: Claude Code를 활용한 금융 AI 시스템 구축 사례
- **GARP 제출**: FRM 자격과 AI 리스크 관리를 결합한 실무 논문
- **금감원 규제 검토**: AI 가이드라인 수립 참고자료 제공

규제 및 제도 대응

- **금감원 AI 기본법 컴플라이언스 검토 요청**: AI 기본법 시행령 및 가이드라인이 수립되는 시점에 맞추어, 본 시스템의 설명 가능성 프레임워크에 대한 검토를 요청할 계획이다.

후속 작업

- **온프레임 운영 데이터 결과**: 실제 운영 데이터에서의 성능 결과를 논문 보충 자료로 추가
- **공개 GitHub 저장소**: 조직 정보를 제거한 sanitized 버전의 코드를 공개 저장소로 공개

이 프로젝트는 “자원의 부족”이 아니라 “자원의 재정의”를 통해 완성되었다.
데스크톱 GPU 1대와 AI 에이전트들의 조합이 전용 인프라를 대체할 수 있음을 보여준 사례다.

이 문서는 Claude Code(Anthropic)의 도움을 받아 작성되었습니다. 아키텍처 결정, 실험 설계, 결과 해석은 인간 저자가 주도하였으며, Claude Code는 코드 구현, 문서 초안 작성, 실험 실행을 보조하였습니다.